

模式识别 (Pattern Recognition)

期末复习笔记

整理人: Your Name

2026 年 1 月 19 日

目录

1 绪论 (3 学时向世明)	3
1.1 模式识别基础	3
1.2 模式识别系统流程	3
1.3 模式识别系统设计	3
1.4 模式识别方法分类	4
1.5 本课程内容体系	4
2 贝叶斯决策理论 (3 学时向世明)	5
2.1 核心公式	5
2.2 最小错误率贝叶斯决策	5
2.3 最小风险贝叶斯决策	5
2.4 带拒识的贝叶斯决策 (Reject Option)	6
2.5 开放集识别 (Open Set Recognition)	6
2.6 分类器设计	7
2.7 高斯密度下的判别函数	7
2.7.1 情形一: $\Sigma_i = \sigma^2 I$ (各特征独立且方差相等)	8
2.7.2 情形二: $\Sigma_i = \Sigma$ (协方差矩阵相等)	8
2.7.3 情形三: $\Sigma_i \neq \Sigma_j$ (协方差矩阵任意)	9
2.8 错误率分析	9
2.9 离散特征贝叶斯分类	10
2.9.1 独立二值特征 (Independent Binary Features)	10
2.10 复合模式分类 (Compound Decision)	11
2.11 多分类器融合 (Multi-classifier Fusion)	11
2.12 模式分类与特征的关系	12
3 概率密度函数估计 (6 学时张燕明)	13
3.1 基本概念	13
3.2 最大似然估计 (Maximum Likelihood Estimation, MLE)	13

3.3 贝叶斯估计 (Bayesian Estimation)	13
补充笔记: 贝叶斯估计公式详解	14
3.4 正态分布下的贝叶斯估计	16
3.5 贝叶斯学习 (Bayesian Learning)	16
3.5.1 1. 迭代更新公式	16
3.5.2 2. 实例: 均匀分布边界的估计	16
3.6 特征维数问题	18
3.7 期望最大化 (EM) 算法	18
3.7.1 1. 经典案例: 三硬币模型 (The Three Coins Example)	18
3.7.2 2. EM 算法在高斯混合模型 (GMM) 中的应用	19
3.7.3 3. EM 算法总结	20
3.8 隐马尔可夫模型 (HMM) 详解与实例	20
3.8.1 1. 具体的模型定义与矩阵实例	20
3.8.2 2. 状态转移图 (State Transition Diagram)	21
3.8.3 3. HMM 的三个核心问题推导	21
4 非参数法 (Non-parametric Methods) (3 学时张燕明)	23
4.1 1. 引言: 为什么要用非参数法?	23
4.2 2. 概率密度估计的基本原理	23
4.3 3. Parzen 窗估计 (核密度估计)	23
4.4 4. k_n -近邻估计 (k-NN Estimation)	24
4.5 5. 最近邻分类器 (Nearest Neighbor Rule)	25
4.6 6. 计算复杂度的优化	25
4.7 7. 距离度量与切距离	26
附录: 常用 LaTeX 模板	27

1 绪论 (3 学时向世明)

1.1 模式识别基础

- 基本定义：
 - 模式 (**Pattern**): 对客体 (Object) 或事件 (Event) 的描述, 通常是向量形式。
 - 模式类 (**Pattern Class**): 具有共同属性的模式集合。
 - 模式识别: 利用机器 (计算机) 自动将特定模式映射到特定类别的过程。
- 经典实例:
 - 硬币分类: 利用尺寸、重量等特征区分不同面值的硬币。
 - 鱼的分类 (Duda 教材经典例子): 区分“三文鱼 (Salmon)”和“鲈鱼 (Sea Bass)”。单一特征 (如长度) 往往重叠严重, 需要多特征 (如长度 + 光泽度) 组合以寻找最佳决策边界。

1.2 模式识别系统流程

一个典型的模式识别系统包含以下五个核心环节:

1. 数据获取 (**Data Acquisition**): 通过传感器将物理变量转换为数字信号 (如摄像头采集图像、麦克风采集声音)。
2. 预处理 (**Preprocessing**): 去噪、分割、平滑、归一化, 旨在提高信噪比, 简化后续处理。
3. 特征提取与选择 (**Feature Extraction & Selection**):
 - 提取: 从原始数据中计算新的特征 (变换)。
 - 选择: 从一组特征中挑选出最具区分力的子集 (降维), 避免“维数灾难”。
4. 分类决策 (**Classification/Decision**): 利用训练好的模型 (分类器) 对输入特征进行分类判定。
5. 后处理 (**Post-processing**): 利用上下文信息 (Context)、先验知识或代价函数对分类结果进行修正 (例如在 OCR 中利用语言模型纠错)。

1.3 模式识别系统设计

设计周期通常是一个不断迭代的反馈过程:

- 数据收集: 划分训练集 (Training Set)、验证集 (Validation Set) 和测试集 (Test Set)。
- 特征选择: 寻找具有不变性 (**Invariance**) 的特征 (如对平移、旋转、缩放不敏感)。
- 模型选择: 决定使用统计模型、神经网络模型还是句法模型; 决定模型的复杂度。
- 训练 (**Training**): 利用样本进行学习, 估计模型参数。

- **评价 (Evaluation):** 核心指标是泛化能力 (**Generalization**), 即在未见过的测试数据上的表现, 需警惕过拟合 (**Overfitting**)。

1.4 模式识别方法分类

基于数据形式 :

- **统计模式识别 (Statistical PR):** 基于特征向量, 利用概率论和统计学理论 (本课程重点)。
- **句法/结构模式识别 (Syntactic/Structural PR):** 将模式分解为基元 (Primitives), 利用形式语言和语法规则描述结构关系 (适合汉字、染色体等结构性强的对象)。

基于学习方式 :

- **监督学习 (Supervised Learning):** 训练数据带有类别标签 (Label)。
- **无监督学习 (Unsupervised Learning):** 训练数据无标签, 旨在发现数据内部结构 (如聚类)。
- **强化学习 (Reinforcement Learning):** 通过与环境交互, 基于奖励/惩罚机制进行学习。

1.5 本课程内容体系

- **基础:** 贝叶斯决策理论 (统计模式识别的基石)。
- **估计:** 概率密度函数估计 (参数法与非参数法)。
- **分类器:** 线性分类器、非线性分类器 (神经网络、SVM、决策树)。
- **工程:** 特征提取与选择、聚类分析。

2 贝叶斯决策理论 (3 学时向世明)

2.1 核心公式

首先, 我们需要充分理解先验、似然、后验的意义。在贝叶斯决策理论中, 最重要的公式是贝叶斯公式:

$$P(\omega_i|x) = \frac{p(x|\omega_i)P(\omega_i)}{p(x)} \quad (1)$$

其中, $P(\omega_i)$ 是类别 ω_i 的先验概率, $p(x|\omega_i)$ 是在类别 ω_i 下观测到样本 x 的似然概率密度, $P(\omega_i|x)$ 是在观测到样本 x 后类别 ω_i 的后验概率。

一个典型案例是 HIV 检测, 假设某种检测方法对 HIV 阳性患者的检测准确率为 99.9%, 对 HIV 阴性患者的检测准确率为 99.5%。如果一个人被检测为阳性, 我们需要计算他实际患有 HIV 的概率。假设总体中 HIV 阳性率为 0.1%, 则可以利用贝叶斯公式计算后验概率。概率计算为:

$$P(\text{HIV}^+|\text{Test}^+) = \frac{P(\text{Test}^+|\text{HIV}^+)P(\text{HIV}^+)}{P(\text{Test}^+)} \quad (2)$$

2.2 最小错误率贝叶斯决策

已知先验概率以及类条件概率密度函数, 最小错误率贝叶斯决策等价于最大后验概率决策:

$$\text{Decide } \omega_i \text{ if } P(\omega_i|x) > P(\omega_j|x), \forall j \neq i \quad (3)$$

这比较容易理解, 因为大多数情况下我们总是希望在当前特征下做出最有可能的决策。

2.3 最小风险贝叶斯决策

在实际应用中, 不同类型的错误造成的后果严重程度不同 (例如误诊癌症), 因此引入损失函数 (Loss Function) 的概念。定义 $\lambda(\alpha_i|\omega_j)$ 为当真实类别为 ω_j 时采取决策 α_i 所带来的损失。

我们的目标是最小化期望风险 (Expected Risk)。对于观测样本 x , 采取决策 α_i 的条件风险定义为:

$$E_{\omega|x}[\lambda(\alpha_i|\omega)] = \sum_{j=1}^c \lambda(\alpha_i|\omega_j) \underbrace{P(\omega_j|x)}_{\text{在该条件下}\omega\text{发生的概率}} \quad (4)$$

决策规则: 选择风险最小的决策 a :

$$a = \arg \min_{j=1, \dots, a} R(\alpha_j|x) \quad (5)$$

特例: 0-1 损失函数 如果损失函数定义为 “对则无损, 错则罚 1”:

$$\lambda(\alpha_i|\omega_j) = \begin{cases} 0, & i = j \\ 1, & i \neq j \end{cases} \quad (6)$$

此时, 最小风险决策完全等价于最小错误率贝叶斯决策 (即最大后验概率决策 MAP)。

2.4 带拒识的贝叶斯决策 (Reject Option)

在某些高风险应用（如医疗诊断、指纹识别）中，如果分类器对某个样本的判断信心不足，为了避免做出严重的错误决策，可以选择“拒绝识别”（Reject），交由人工处理。

- **动作空间扩展：**定义决策集合为 $\{\alpha_1, \dots, \alpha_c, \alpha_{c+1}\}$ ，其中 α_{c+1} 表示“拒识”。
- **损失函数定义：**设 λ_s 为误分带来的损失（Substitution cost）， λ_r 为拒识带来的损失（Rejection cost）。通常要求 $0 < \lambda_r < \lambda_s$ （否则没必要拒识）。

$$\lambda(\alpha_i|\omega_j) = \begin{cases} 0, & i = j \quad (\text{正确}) \\ \lambda_s, & i \neq j, i \leq c \quad (\text{误分}) \\ \lambda_r, & i = c + 1 \quad (\text{拒识}) \end{cases} \quad (7)$$

- **条件风险：**

– 做出分类决策 α_i ($i \leq c$) 的风险：

$$R(\alpha_i|x) = \sum_{j \neq i} \lambda_s P(\omega_j|x) = \lambda_s [1 - P(\omega_i|x)]$$

– 做出拒识决策 α_{c+1} 的风险：

$$R(\alpha_{c+1}|x) = \lambda_r$$

- **决策规则：**当且仅当拒识的风险小于分类的最小风险时，选择拒识。

$$\lambda_r < \min_{i \leq c} R(\alpha_i|x) = \lambda_s (1 - \max_i P(\omega_i|x))$$

整理得拒识条件为：

$$\max_i P(\omega_i|x) < 1 - \frac{\lambda_r}{\lambda_s} \quad (8)$$

即：只有当最大后验概率低于阈值 $T = 1 - \frac{\lambda_r}{\lambda_s}$ 时，才拒绝识别。

2.5 开放集识别 (Open Set Recognition)

传统的模式识别通常假设**闭集 (Closed Set)**环境，即测试样本一定属于训练集中定义的 c 个类别之一。然而在实际应用中，可能会出现训练集中从未见过的**未知类别 (Unknown Classes)**。

- **问题定义：**

- **训练集：**包含 c 个已知类别的样本。
- **测试集：**包含已知类别的样本 + 未知类别的样本。
- **目标：**正确分类已知类别样本，同时将未知类别样本识别为“未知”或“第 $c+1$ 类”。

- **与拒识的区别：**

- **拒识**: 是在已知类别中“拿不准”(位于决策边界附近), 属于**模糊区**。
- **开放集**: 是完全不属于已知类别(位于特征空间远离已知分布的区域), 属于**未知区**。

- **简单解决方案(基于阈值)**:

引入距离阈值或概率阈值。

$$\text{Decision}(x) = \begin{cases} \arg \max_i g_i(x), & \text{if } \max_i g_i(x) > \tau \\ \text{Unknown}, & \text{otherwise} \end{cases} \quad (9)$$

其中 $g_i(x)$ 可以是后验概率, 也可以是样本到类中心的距离的负值(如 $1/d(x, \mu_i)$)。这实际上限制了每个类别的决策区域必须是紧致的(Bounded), 而非延伸至无穷远。

2.6 分类器设计

分类器可以看作是一个计算一组**判别函数(Discriminant Functions)** $g_i(x)$ 并选择最大值的机器。

决策规则: 如果 $g_i(x) > g_j(x), \forall j \neq i$, 则将 x 归类为 ω_i 。

判别函数的形式:

1. **基础形式**: 常用的判别函数形式有:

- $g_i(x) = P(\omega_i|x)$ (直接使用后验概率)
- $g_i(x) = p(x|\omega_i)P(\omega_i)$ (去掉分母 $p(x)$)
- $g_i(x) = \ln p(x|\omega_i) + \ln P(\omega_i)$ (取对数, 变乘法为加法)

2. **更一般形式(General Case)**: PPT 中给出了判别函数的更一般定义:

$$g_i(x) = f(p(x|\omega_i)) + h(x, \omega_i) \quad (10)$$

其中:

- $f(\cdot)$ 是单调函数(通常取 $\ln$), 用于变换似然度。
- $h(x, \omega_i)$ 是包含先验概率 $P(\omega_i)$ 和损失 λ 的项。

3. **注: 与神经网络的联系**这与现代神经网络最后的判别器(Discriminator)设计本质一致。神经网络最后一层(Logits)输出的值 $z_i = w_i^T x + b_i$, 实际上就是在拟合这个 $g_i(x)$ 。其中 $w_i^T x$ 对应特征的似然度部分, 而偏置 b_i 往往隐含了先验概率 $P(\omega_i)$ 的信息。

2.7 高斯密度下的判别函数

当类条件概率密度 $p(x|\omega_i)$ 服从多元正态分布 $N(\mu_i, \Sigma_i)$ 时, 判别函数具有解析解。

多元正态分布概率密度公式:

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \quad (11)$$

取对数形式的判别函数 $g_i(x)$ 为:

$$g_i(x) = -\frac{1}{2} (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i) - \frac{1}{2} \ln |\Sigma_i| + \ln P(\omega_i) \quad (12)$$

根据协方差矩阵 Σ_i 的不同, 分为三种情形:

2.7.1 情形一： $\Sigma_i = \sigma^2 I$ （各特征独立且方差相等）

此时各类协方差矩阵相同且为对角阵，几何上数据分布呈正球体，大小相同。判别函数可简化。

1. 先验概率相等 ($P(\omega_i) = P(\omega_j)$):

- 判别依据：此时 $\ln P(\omega_i)$ 项为常数可忽略，决策规则简化为最小化欧氏距离平方：

$$g_i(x) \propto -\|x - \mu_i\|^2$$

- 分类器名称：最小距离分类器 (Minimum Distance Classifier)。
- 决策边界：连接两类均值向量 μ_i 和 μ_j 线段的垂直平分面，经过中点 $x_0 = \frac{1}{2}(\mu_i + \mu_j)$ 。

2. 先验概率不相等 ($P(\omega_i) \neq P(\omega_j)$):

- 判别依据：依然是线性判别函数，但包含偏置项：

$$g_i(x) = \frac{1}{\sigma^2} \mu_i^T x - \frac{1}{2\sigma^2} \mu_i^T \mu_i + \ln P(\omega_i)$$

- 决策边界：决策面依然与 $\mu_i - \mu_j$ 正交，但不再经过连线中点。
- 偏移规律：决策面会向先验概率较小的一侧偏移（即先验概率大的类别占据更大的决策区域）。偏移量为：

$$x_0 = \frac{1}{2}(\mu_i + \mu_j) - \frac{\sigma^2}{\|\mu_i - \mu_j\|^2} \ln \frac{P(\omega_i)}{P(\omega_j)} (\mu_i - \mu_j)$$

2.7.2 情形二： $\Sigma_i = \Sigma$ （协方差矩阵相等）

此时各类数据分布呈超椭球体，形状和方向相同（平行），但中心不同。

1. 先验概率相等 ($P(\omega_i) = P(\omega_j)$):

- 判别依据：最小化马氏距离 (Mahalanobis Distance) 平方：

$$r^2 = (x - \mu_i)^T \Sigma^{-1} (x - \mu_i) \quad (13)$$

- 决策边界：经过均值连线中点的超平面。但由于 Σ 不再是单位阵，决策面通常不与均值连线正交。

2. 先验概率不相等 ($P(\omega_i) \neq P(\omega_j)$):

- 判别依据：线性判别函数 $g_i(x) = w_i^T x + w_{i0}$ ，其中 $w_i = \Sigma^{-1} \mu_i$ 。
- 决策边界：决策面 x_0 同样会偏离中点，向先验概率较小的一方偏移。

2.7.3 情形三: $\Sigma_i \neq \Sigma_j$ (协方差矩阵任意)

- 几何意义: 各类数据的分布形状、方向、大小都不一样。
- 判别函数: 无法消除二次项, 形式为 $x^T W_i x + w_i^T x + w_{i0}$, 是二次判别函数。其中 $\mathbf{W}_i = -\frac{1}{2} \Sigma_i^{-1}$, $\mathbf{w}_i = \Sigma_i^{-1} \boldsymbol{\mu}_i$, $w_{i0} = -\frac{1}{2} \boldsymbol{\mu}_i^T \Sigma_i^{-1} \boldsymbol{\mu}_i - \frac{1}{2} \ln(|\Sigma_i|) + \ln(P(\omega_i))$
- 决策边界: 超二次曲面。根据协方差矩阵的差异, 可能表现为超球面、超椭球面、超双曲面或超平面等多种形态。

• 一个具体例子

— 2类, 2D $P(\omega_1) = P(\omega_2) = 0.5$

$$\boldsymbol{\mu}_1 = \begin{bmatrix} 3 \\ 6 \end{bmatrix}; \quad \Sigma_1 = \begin{pmatrix} 1/2 & 0 \\ 0 & 2 \end{pmatrix} \quad \Sigma_1^{-1} = \begin{pmatrix} 2 & 0 \\ 0 & 1/2 \end{pmatrix}$$

$$\boldsymbol{\mu}_2 = \begin{bmatrix} 3 \\ -2 \end{bmatrix}; \quad \Sigma_2 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \quad \Sigma_2^{-1} = \begin{pmatrix} 1/2 & 0 \\ 0 & 1/2 \end{pmatrix}$$

— 决策面 $g_1(\mathbf{x}) = g_2(\mathbf{x})$

$$x_2 = 3.514 - 1.125x_1 + 0.1875x_1^2$$

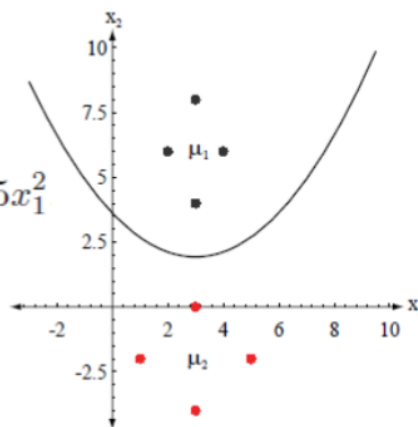


图 1: 贝叶斯判别例子

2.8 错误率分析

贝叶斯错误率 (Bayes Error) 是理论上分类器能达到的最低错误率。

$$P(\text{error}) = \int P(\text{error}|x)p(x)dx \quad (14)$$

对于两类问题, 错误率如下:

$$\begin{aligned} P(\text{error}) &= P(\mathbf{x} \in R_2, \omega_1) + P(\mathbf{x} \in R_1, \omega_2) \\ &= \int_{R_2} p(\mathbf{x} | \omega_1) P(\omega_1) d\mathbf{x} + \int_{R_1} p(\mathbf{x} | \omega_2) P(\omega_2) d\mathbf{x} \\ &= P(\mathbf{x} \in R_2 | \omega_1) P(\omega_1) + P(\mathbf{x} \in R_1 | \omega_2) P(\omega_2) \end{aligned} \quad (15)$$

多类情形如下:

$$P(\text{error}) = \sum_{i=1}^c \sum_{j \neq i} P(\mathbf{x} \in R_j, \omega_i) \quad (16)$$

2.9 离散特征贝叶斯分类

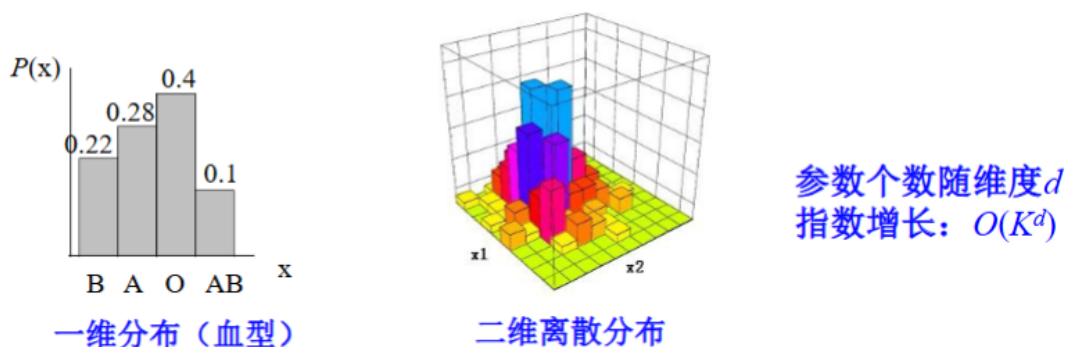


图 2: 离散分布图

当特征向量 $x = [x_1, \dots, x_d]^T$ 的分量是离散值（如性别、血型、二值化图像）而非连续值时，我们需要使用概率质量函数（PMF）而非概率密度函数。

- **维数灾难**: 如果每个特征 x_j 有 L 个可能的取值，那么 d 维特征空间的总状态数为 L^d 。要估计每个状态的条件概率 $P(x|\omega_i)$ ，需要的样本量呈指数级增长。
- **朴素贝叶斯假设 (Naïve Bayes Assumption)**: 为了简化估计，假设在给定类别 ω_i 的条件下，各个特征分量 x_j 之间是相互独立的。

$$P(x|\omega_i) = \prod_{j=1}^d P(x_j|\omega_i) \quad (17)$$

虽然这个假设在现实中往往不成立，但朴素贝叶斯分类器在实际应用中（如文本分类）常表现出惊人的有效性。

2.9.1 独立二值特征 (Independent Binary Features)

这是一个特例，假设每个特征 $x_j \in \{0, 1\}$ （伯努利分布）。设 $p_{ij} = P(x_j = 1|\omega_i)$ ，即在 ω_i 类中第 j 个特征为 1 的概率。

- **似然函数**:

$$P(x|\omega_i) = \prod_{j=1}^d p_{ij}^{x_j} (1 - p_{ij})^{1-x_j} \quad (18)$$

- **判别函数推导**: 取对数似然比 $g(x) = \ln \frac{P(x|\omega_1)}{P(x|\omega_2)} + \ln \frac{P(\omega_1)}{P(\omega_2)}$ 。代入后可整理为线性判别函数形式:

$$g(x) = \sum_{j=1}^d w_j x_j + w_0 \quad (19)$$

其中权重 w_j 和偏置 w_0 分别为:

$$w_j = \ln \frac{p_{1j}(1 - p_{2j})}{p_{2j}(1 - p_{1j})}, \quad w_0 = \sum_{j=1}^d \ln \frac{1 - p_{1j}}{1 - p_{2j}} + \ln \frac{P(\omega_1)}{P(\omega_2)} \quad (20)$$

- **结论：**对于独立二值特征，贝叶斯分类器的决策边界是一个**超平面**（线性分类器）。

下面是一个三维二值分布的例子。

- **一个例子: 3D binary data**

- $P(\omega_1)=0.5, P(\omega_2)=0.5$

- $p_i=0.8, q_i=0.5, \quad i=1,2,3$

$$P(\mathbf{x}|\omega_1) = \prod_{i=1}^d p_i^{x_i} (1-p_i)^{1-x_i} \quad P(\mathbf{x}|\omega_2) = \prod_{i=1}^d q_i^{x_i} (1-q_i)^{1-x_i}$$

$$g(\mathbf{x}) = \sum_{i=1}^d w_i x_i + w_0$$

$$w_i = \ln \frac{.8(1-.5)}{.5(1-.8)} = 1.3863$$

$$w_0 = \sum_{i=1}^3 \ln \frac{1-.8}{1-.5} + \ln \frac{.5}{.5} = -2.7489$$

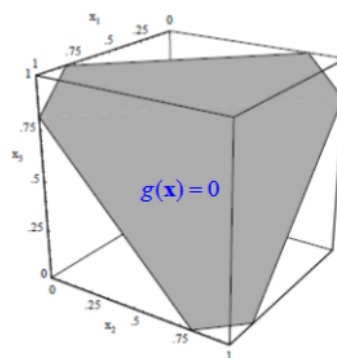


图 3: 3D binary example

2.10 复合模式分类 (Compound Decision)

当我们不仅是对单个样本分类，而是要对一连串相关的样本 $X = (x_1, \dots, x_n)$ 进行分类（例如语音识别、手写体识别）时，需要考虑上下文信息。

- **目标：**找到最优的类别序列 $\omega = (\omega_1, \dots, \omega_n)$ 使得后验概率 $P(\omega|X)$ 最大。
- **马尔可夫假设：**

1. 观测独立性: $p(X|\omega) = \prod p(x_t|\omega_t)$

2. 状态转移（一阶马尔可夫链）: $P(\omega_t|\omega_{t-1}, \dots) = P(\omega_t|\omega_{t-1})$

这构成了隐马尔可夫模型 (HMM) 的基础。

2.11 多分类器融合 (Multi-classifier Fusion)

当对于同一个分类问题有 K 个不同的分类器时，我们可以利用贝叶斯方法将它们的结果进行融合，以期获得比单一分类器更好的性能。

- **基本思路：**将 K 个分类器的输出 $e_1, e_2, \dots, e_K$ 看作是样本 x 的一个新的特征向量 $\mathbf{e} = [e_1, e_2, \dots, e_K]^T$ 。

- **融合决策**：在新的特征空间中应用贝叶斯公式计算后验概率：

$$P(\omega_i|\mathbf{e}) = \frac{p(\mathbf{e}|\omega_i)P(\omega_i)}{p(\mathbf{e})} \quad (21)$$

其中 $p(\mathbf{e}|\omega_i) = p(e_1, \dots, e_K|\omega_i)$ 是联合类条件概率密度。

- **独立性假设**：直接估计联合概率 $p(e_1, \dots, e_K|\omega_i)$ 需要的样本量随分类器数量指数级增加。通常假设各分类器在给定类别下是条件独立的（Naïve Bayes 假设）：

$$p(e_1, \dots, e_K|\omega_i) \approx \prod_{k=1}^K p(e_k|\omega_i) \quad (22)$$

这样可以将复杂的高维估计问题简化为一维估计问题。

2.12 模式分类与特征的关系

- **贝叶斯错误率 (Bayes Error Rate)**:

- 对于给定的特征空间，贝叶斯错误率是分类错误率的**理论下界**。它由特征空间中类条件概率密度的重叠程度决定，与具体的分类器设计无关。
- 想要降低贝叶斯错误率，唯一的方法是**改进特征空间**（如寻找更有区分力的特征、增加新特征）。

- **特征维数的影响**:

- **理想情况**：增加特征维数通常会增加类间距离（如高斯分布下的马氏距离 r^2 ），从而降低贝叶斯错误率。例如，数据在 1D 投影上重叠，但在 3D 空间中可能完全可分。
- **实际情况**：在训练样本有限的情况下，增加特征维数并不总能降低错误率。

- **过拟合 (Overfitting)**:

- **现象**：模型在训练集上表现完美（如用高阶多项式拟合），但在测试集上误差很大。
- **原因**：特征维数过高相对于样本数过少，导致参数估计（如协方差矩阵）不准确。例如，估计 d 维协方差矩阵至少需要 $d+1$ 个样本，要获得准确估计则需要更多。
- **对策**:
 1. **特征降维**：特征提取（PCA, LDA）或特征选择。
 2. **正则化 (Regularization / Shrinkage)**：对参数估计进行平滑。例如正则化判别分析 (RDA)，将协方差矩阵向单位阵或公共协方差矩阵收缩：

$$\hat{\Sigma}_i(\alpha) = (1 - \alpha)\hat{\Sigma}_i + \alpha\mathbf{I} \quad (23)$$

3 概率密度函数估计 (6 学时张燕明)

3.1 基本概念

- **参数估计 (Parametric Estimation):** 假设概率密度函数 $p(x|\theta)$ 的形式已知 (如正态分布), 但其中的参数 θ 未知。任务是利用样本集估计这些参数。
- **非参数估计 (Non-parametric Estimation):** 不假设概率密度函数的数学形式, 直接由样本估计概率密度函数本身。
- **统计量 (Statistic):** 样本的某种函数 $d(x_1, \dots, x_n)$, 用于推断总体信息 (如样本均值)。
- **估计类型:**
 - **点估计 (Point Estimation):** 构造一个统计量作为参数的估计值 (如 $\hat{\mu}$)。
 - **区间估计 (Interval Estimation):** 确定参数的一个可能取值范围 (置信区间)。

3.2 最大似然估计 (Maximum Likelihood Estimation, MLE)

- **基本原理:** 假设样本 $D = \{x_1, \dots, x_n\}$ 是独立同分布 (i.i.d.) 的。最大似然估计的目标是找到参数 $\hat{\theta}$, 使得观测到当前样本集的概率 (似然) 最大。

- **似然函数:**

$$l(\theta) = p(D|\theta) = \prod_{i=1}^n p(x_i|\theta) \quad (24)$$

- **对数似然函数:** 为了计算方便 (将乘法转为加法), 通常最大化对数似然:

$$H(\theta) = \ln l(\theta) = \sum_{i=1}^n \ln p(x_i|\theta) \quad (25)$$

- **求解方法:** 令梯度为 0: $\nabla_{\theta} H(\theta) = 0$ 。
- **高斯分布下的 MLE:** 对于 $N(\mu, \Sigma)$, 其参数估计为:

- 均值: $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$ (样本均值)
- 协方差: $\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})(x_i - \hat{\mu})^T$ (样本协方差, 注意分母为 n 而非 $n-1$)

3.3 贝叶斯估计 (Bayesian Estimation)

- **核心思想:** 与极大似然估计不同, 贝叶斯估计将参数 θ 视为一个随机变量。
- **如何使用 $p(\theta|D)$ 获得关于数据的分布?** PPT 中给出了以下两种主要方法:

方法一：最大后验估计 (Maximum A Posteriori, MAP)

- 基本原理：选择使后验概率 $p(\theta|D)$ 取值最大的参数 θ 作为估计值。
- 数学表达式：

$$\hat{\theta}_{MAP} = \arg \max_{\theta} p(\theta|D) = \arg \max_{\theta} [p(D|\theta)p(\theta)] \quad (26)$$

通过取对数，可以转化为：

$$\hat{\theta}_{MAP} = \arg \max_{\theta} [\ln p(D|\theta) + \ln p(\theta)] \quad (27)$$

- 物理意义：在似然函数的基础上引入了先验信息。
- 联系：机器学习中普遍使用的 L_2 正则化（权重衰减），在统计学上等价于假设参数 θ 服从高斯先验 $N(0, I)$ 后的 MAP 估计。

方法二：数据的后验分布 (The Posterior Distribution of Data)

这是完整的贝叶斯方法，不把参数固定为某一个点，而是考虑参数的所有可能性。

- 基本原理：利用全概率公式，对给定参数 θ 下的概率密度进行加权平均，权重即为参数的后验概率 $p(\theta|D)$ 。
- 预测公式：

$$p(x|D) = \int_{\theta} p(x|\theta, D)p(\theta|D)d\theta = \int_{\theta} p(x|\theta)p(\theta|D)d\theta \quad (28)$$

（注：假设给定参数 θ 时， x 的分布与训练集 D 无关）。

- 物理意义：这是不同参数下的密度函数的加权平均。
- 近似计算：当积分难以直接计算时，常用 M 个采样点的平均来近似：

$$p(x|D) \approx \frac{1}{M} \sum_{i=1}^M p(x|\theta_i) \quad (29)$$

考点/重点: 重点对比

- MAP 最终得到的是一个确定的参数值 $\hat{\theta}$ ，预测时使用 $p(x|\hat{\theta})$ 。
- 数据的后验分布最终得到的是一个完整的概率分布 $p(x|D)$ ，它综合了参数空间中所有可能的模型。

补充笔记：贝叶斯估计公式详解

贝叶斯估计的核心预测公式为：

$$p(x|D) = \int \underbrace{p(x|\theta)}_{\text{第一项}} \underbrace{p(\theta|D)}_{\text{第二项}} d\theta \quad (30)$$

这两项的物理意义和获取方式完全不同，详细辨析如下：

1. 第一项 $p(x|\theta)$ ——模型假设（已知形式）

这一项代表“在参数 θ 确定的情况下，样本 x 服从什么分布”。

- 来源：人为假设 / 先验知识。在做贝叶斯估计之前，必须先选定一个概率模型（如正态分布、指数分布等）。
- 获取方式：直接写出概率密度函数的数学公式。
- 例子：若假设数据服从正态分布 $N(\mu, \sigma^2)$ （设 σ^2 已知， $\theta = \mu$ ），则第一项就是正态分布的密度公式：

$$p(x|\mu) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (31)$$

注意：在此处 x 是自变量， μ 被视为暂时的常数。

2. 第二项 $p(\theta|D)$ ——参数后验（计算核心）

这一项代表“在观察到训练集 D 之后，参数 θ 取各种值的可能性是多少”。这是贝叶斯估计计算最繁琐的一步。

- 来源：通过 贝叶斯公式 计算得出。

- 计算公式：

$$p(\theta|D) = \frac{p(D|\theta)P(\theta)}{p(D)} = \frac{(\prod_{k=1}^n p(x_k|\theta)) P(\theta)}{\int (\prod_{k=1}^n p(x_k|\theta)) P(\theta)d\theta} \quad (32)$$

- 计算步骤：

1. 确定先验分布 $P(\theta)$ ：根据经验假设 θ 的分布（例如：假设均值 μ 本身也服从一个正态分布 $N(\mu_0, \sigma_0^2)$ ）。
2. 计算联合似然 $p(D|\theta)$ ：利用独立同分布 (i.i.d.) 假设，将训练集中所有样本的概率乘起来：

$$p(D|\theta) = \prod_{k=1}^n p(x_k|\theta)$$

（这里用到的 $p(x_k|\theta)$ 就是第一项里的那个公式）。

3. 相乘并归一化：

$$\text{后验} \propto \text{似然} \times \text{先验}$$

计算乘积后，除以归一化因子（分母），得到最终的参数分布函数。

考点/重点: 总结

- $p(x|\theta)$ ：是你选的（模型假设，直接写公式）。
- $p(\theta|D)$ ：是你算的（利用数据 D 更新先验 $P(\theta)$ 得到的参数分布）。

3.4 正态分布下的贝叶斯估计

以一维正态分布为例，假设方差 σ^2 已知，估计均值 μ 。设 μ 的先验分布为 $N(\mu_0, \sigma_0^2)$ 。

- 参数后验分布： $p(\mu|D)$ 仍为正态分布 $N(\mu_n, \sigma_n^2)$ 。

$$\mu_n = \frac{n\sigma_0^2}{n\sigma_0^2 + \sigma^2} \hat{\mu}_n + \frac{\sigma^2}{n\sigma_0^2 + \sigma^2} \mu_0 \quad (33)$$

$$\sigma_n^2 = \frac{\sigma_0^2 \sigma^2}{n\sigma_0^2 + \sigma^2} \quad (34)$$

其中 $\hat{\mu}_n$ 是样本均值。可以看出，后验均值 μ_n 是样本均值和先验均值的加权和。

- 性质：

- 当 $n \rightarrow \infty$ 时， $\mu_n \rightarrow \hat{\mu}_n$ ， $\sigma_n^2 \rightarrow 0$ 。参数分布收敛于一个尖峰（Dirac delta 函数），贝叶斯估计趋近于最大似然估计。
- 概率密度估计结果 $p(x|D)$ 是 $N(\mu_n, \sigma^2 + \sigma_n^2)$ ，方差包含了数据噪声 σ^2 和参数估计的不确定性 σ_n^2 。

3.5 贝叶斯学习 (Bayesian Learning)

贝叶斯估计的一个重要特性是支持**增量学习 (Incremental Learning)**。随着样本逐个到来，参数的后验分布可以不断更新，且不需要重新计算之前的所有数据。

3.5.1 1. 迭代更新公式

记 $D^n = \{x_1, x_2, \dots, x_n\}$ 为包含 n 个样本的集合。根据贝叶斯公式，参数 θ 的后验分布可以表示为（注意这个表示需要一定的技巧，可以稍微背一背）：

$$p(\theta|D^n) = \frac{p(x_n|\theta)p(\theta|D^{n-1})}{\int p(x_n|\theta)p(\theta|D^{n-1})d\theta} \quad (35)$$

即：

$$\underbrace{p(\theta|D^n)}_{\text{当前后验}} \propto \underbrace{p(x_n|\theta)}_{\text{当前似然}} \cdot \underbrace{p(\theta|D^{n-1})}_{\text{当前先验 (即上一时刻的后验)}} \quad (36)$$

这意味着，上一时刻的后验分布 $p(\theta|D^{n-1})$ 直接作为当前时刻的先验分布。随着 $n \rightarrow \infty$ ，后验分布 $p(\theta|D^n)$ 通常会越来越尖锐，最终收敛于参数真值附近的 Dirac δ 函数。

3.5.2 2. 实例：均匀分布边界的估计

问题描述：假设样本 x 服从均匀分布 $U(0, \theta)$ ，即 $p(x|\theta) = 1/\theta$ ($0 \leq x \leq \theta$)，否则为 0。已知先验分布 $p(\theta) = U(0, 10)$ 。现有观测样本序列 $D = \{4, 7, 2, 8\}$ ，试利用贝叶斯学习迭代估计 θ 。

迭代过程：

1. 初始状态 (D^0):

$$p(\theta|D^0) = p(\theta) = \frac{1}{10}, \quad 0 \leq \theta \leq 10$$

2. 样本 $x_1 = 4$ 到来:

$$p(\theta|D^1) \propto p(x_1|\theta)p(\theta|D^0) = \frac{1}{\theta} \cdot \frac{1}{10}$$

由于必须满足 $\theta \geq x_1 = 4$, 故有效区间为 $4 \leq \theta \leq 10$ 。

$$p(\theta|D^1) \propto \frac{1}{\theta}, \quad 4 \leq \theta \leq 10$$

3. 样本 $x_2 = 7$ 到来:

$$p(\theta|D^2) \propto p(x_2|\theta)p(\theta|D^1) \propto \frac{1}{\theta} \cdot \frac{1}{\theta} = \frac{1}{\theta^2}$$

约束条件更新为 $\theta \geq \max(4, 7) = 7$ 。

$$p(\theta|D^2) \propto \frac{1}{\theta^2}, \quad 7 \leq \theta \leq 10$$

4. 样本 $x_3 = 2$ 到来:

$$p(\theta|D^3) \propto p(x_3|\theta)p(\theta|D^2) \propto \frac{1}{\theta} \cdot \frac{1}{\theta^2} = \frac{1}{\theta^3}$$

约束条件 $\theta \geq \max(7, 2) = 7$ 不变。

$$p(\theta|D^3) \propto \frac{1}{\theta^3}, \quad 7 \leq \theta \leq 10$$

5. 样本 $x_4 = 8$ 到来:

$$p(\theta|D^4) \propto p(x_4|\theta)p(\theta|D^3) \propto \frac{1}{\theta} \cdot \frac{1}{\theta^3} = \frac{1}{\theta^4}$$

约束条件更新为 $\theta \geq \max(7, 8) = 8$ 。

$$p(\theta|D^4) = \alpha \cdot \frac{1}{\theta^4}, \quad 8 \leq \theta \leq 10$$

归一化与最终结果: 计算归一化常数 α :

$$\frac{1}{\alpha} = \int_8^{10} \frac{1}{\theta^4} d\theta = \left[-\frac{1}{3\theta^3} \right]_8^{10} = \frac{1}{3} \left(\frac{1}{512} - \frac{1}{1000} \right) \approx \frac{1}{3147.5}$$

因此, 参数 θ 的最终后验分布为:

$$p(\theta|D^4) = \begin{cases} \frac{3147.5}{\theta^4}, & 8 \leq \theta \leq 10 \\ 0, & \text{otherwise} \end{cases} \quad (37)$$

数据的后验分布 (Posterior Predictive Distribution):

$$p(x|D) = \int p(x|\theta)p(\theta|D)d\theta = \int_{\max(x,8)}^{10} \frac{1}{\theta} \cdot \frac{3147.5}{\theta^4} d\theta$$

最终计算结果为:

$$p(x|D) = \begin{cases} 0.1134, & 0 \leq x \leq 8 \\ 786.9(\frac{1}{x^4} - \frac{1}{10^4}), & 8 < x \leq 10 \end{cases} \quad (38)$$

3.6 特征维数问题

- **过拟合 (Overfitting)**: 模型在训练集上表现极好, 但在测试集上性能不佳。通常由于模型过于复杂 (如高阶多项式拟合) 或训练数据相对于特征维数过少导致。
- **维数灾难**: 随着特征维数增加, 为了维持同样的估计精度, 所需样本量呈指数级增长。
- **解决方法**:
 - **特征降维**: 特征选择或提取 (如 PCA)。
 - **正则化 (Regularization/Shrinkage)**: 例如在估计协方差矩阵时, 使用 $\Sigma(\alpha) = (1 - \alpha)\hat{\Sigma} + \alpha I$ 进行平滑。

3.7 期望最大化 (EM) 算法

EM 算法是一种迭代算法, 用于含有隐变量 (Latent Variables) 的概率模型参数的极大似然估计。

3.7.1 1. 经典案例: 三硬币模型 (The Three Coins Example)

问题描述: 假设有 3 枚硬币 A、B、C, 正面朝上的概率分别为 π, p, q 。进行如下实验:

1. 先掷硬币 C。若正面朝上选 A, 若反面朝上选 B。
2. 掷选出的硬币 (A 或 B), 记录结果 (正面记为 1, 反面记为 0)。

重复 n 次实验, 观测数据为 $Y = (y_1, y_2, \dots, y_n)$, 但我们不知道每次具体选的是 A 还是 B (即隐变量 Z 未知)。目标是估计参数 $\theta = (\pi, p, q)$ 。

EM 算法求解过程:

- **E 步 (计算隐变量的期望)**: 在当前参数 $\theta^{(i)}$ 下, 计算第 j 次实验是硬币 A 抛出的概率 (即隐变量的后验概率):

$$\mu_j^{(i+1)} = P(z_j = A | y_j, \theta^{(i)}) = \frac{\pi^{(i)}(p^{(i)})^{y_j}(1-p^{(i)})^{1-y_j}}{\pi^{(i)}(p^{(i)})^{y_j}(1-p^{(i)})^{1-y_j} + (1-\pi^{(i)})(q^{(i)})^{y_j}(1-q^{(i)})^{1-y_j}} \quad (39)$$

- **M 步 (最大化似然更新参数)**: 基于 E 步计算出的概率权重 μ_j , 重新估计参数:

$$\begin{aligned} \pi^{(i+1)} &= \frac{1}{n} \sum_{j=1}^n \mu_j^{(i+1)} \\ p^{(i+1)} &= \frac{\sum_{j=1}^n \mu_j^{(i+1)} y_j}{\sum_{j=1}^n \mu_j^{(i+1)}} \\ q^{(i+1)} &= \frac{\sum_{j=1}^n (1 - \mu_j^{(i+1)}) y_j}{\sum_{j=1}^n (1 - \mu_j^{(i+1)})} \end{aligned} \quad (40)$$

3.7.2 2. EM 算法在高斯混合模型 (GMM) 中的应用

A. 什么是 GMM 与隐变量？ 高斯混合模型 (GMM) 是指由 K 个高斯分布 (Gaussian Distributions) 线性叠加而成的概率模型。其概率密度函数定义为：

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k) \quad (41)$$

其中：

- π_k 是混合系数 (Mixing Coefficient)，且满足 $\sum \pi_k = 1, \pi_k \geq 0$ 。
- $\mathcal{N}(x|\mu_k, \Sigma_k)$ 是第 k 个高斯分量 (Component)。

隐变量 (Latent Variable) 是什么？

在 GMM 的生成过程中，我们可以想象存在一个“上帝掷骰子”的过程：

1. 这是一个 K 面骰子。上帝首先根据概率 π 掷出骰子，决定选择第 k 个高斯分布。这个选择的结果就是隐变量，记为 z (通常用 one-hot 向量表示)。
2. 既然选定了第 k 个分布，就从 $\mathcal{N}(\mu_k, \Sigma_k)$ 中采样生成观测数据 x 。

为什么它是“隐”的？

因为我们只能观测到最终的数据 x (比如班里学生的身高)，但我们不知道这个 x 具体是由哪个分布生成的 (不知道该学生具体属于哪个班级)。EM 算法的目标就是：推断每个数据 x 来自哪个分布 (E 步)，并据此反推这些分布的参数 (M 步)。

B. EM 算法求解步骤 假设观测数据 $X = \{x_1, \dots, x_N\}$ ，模型参数为 $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ 。

1. E 步：计算响应度 (Responsibility)

这一步相当于我们在做“软分类”。我们需要计算在当前参数下，第 n 个观测数据 x_n 属于第 k 个高斯分量的后验概率，记为 $\gamma(z_{nk})$ ：

$$\gamma(z_{nk}) = P(z_k = 1|x_n) = \frac{\overbrace{\pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}^{\text{分量 } k \text{ 产生该数据的可能性}}}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)} \quad (42)$$

直观理解：如果 x_n 离第 k 个高斯中心的距离 μ_k 很近，那么该分量对这个点的“响应度” $\gamma(z_{nk})$ 就会很高 (接近 1)；反之则接近 0。

2. M 步：参数更新

既然我们不知道每个点确切属于哪个组，我们就用 E 步算出的“响应度”作为权重，来对参数进行加权更新。

首先定义第 k 个分量的有效样本总数 N_k ：

$$N_k = \sum_{n=1}^N \gamma(z_{nk}) \quad (43)$$

注：这相当于把所有数据点分给第 k 类的那部分“份额”加起来。

更新公式如下：

- 更新均值 μ_k (加权平均中心):

$$\mu_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) x_n \quad (44)$$

解释: 谁对第 k 类响应度高, 谁就在计算中心时拥有更大的话语权。

- 更新协方差 Σ_k (加权离散程度):

$$\Sigma_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma(z_{nk}) (x_n - \mu_k^{new})(x_n - \mu_k^{new})^T \quad (45)$$

- 更新混合系数 π_k (该类的比重):

$$\pi_k^{new} = \frac{N_k}{N} \quad (46)$$

解释: 第 k 类的有效样本数占总样本数的比例。

重复 E 步和 M 步直到似然函数收敛。

3.7.3 3. EM 算法总结

- **核心思想:** 将复杂的不完全数据似然最大化问题, 转化为简单的完全数据似然期望最大化问题。
- **收敛性:** EM 算法保证每次迭代后似然函数值非递减 ($L(\theta^{(t+1)}) \geq L(\theta^{(t)})$), 但不能保证收敛到全局最优 (容易陷入局部最优, 且对初值敏感)。
- **与 K-Means 的关系:** K-Means 可以看作是 GMM 的一种特例 (硬分配, 协方差矩阵为单位阵且趋于 0 时)。

3.8 隐马尔可夫模型 (HMM) 详解与实例

HMM 是处理序列数据的统计模型。为了理解它, 我们使用一个经典的“盒子摸球”模型作为贯穿始终的例子。

3.8.1 1. 具体的模型定义与矩阵实例

假设我们要模拟一个摸球实验:

- **隐状态 (Hidden States) $Q = \{1, 2, 3\}$:** 我们有 3 个盒子, 分别编号 1, 2, 3。我们看不见手里拿的是哪个盒子。
- **观测值 (Observations) $V = \{, \}$:** 我们每次只能看到从盒子里摸出来的球的颜色。

一个完整的 HMM 模型 $\lambda = (\pi, A, B)$ 由以下具体参数定义:

- (1) **初始状态分布 π (Initial Distribution)** 刚开始实验时, 选中各个盒子的概率:

$$\pi = \begin{bmatrix} 0.2 \\ 0.4 \\ 0.4 \end{bmatrix} \quad (\text{即: 选中盒子 1 的概率是 0.2, 盒子 2 是 0.4...}) \quad (47)$$

(2) 状态转移矩阵 A (Transition Matrix) 如果我们当前在由盒子 i 摸球，下一次换到盒子 j 的概率。这是 HMM 的核心 (马尔可夫链):

$$A = \begin{bmatrix} 0.5 & 0.2 & 0.3 \\ 0.3 & 0.5 & 0.2 \\ 0.2 & 0.3 & 0.5 \end{bmatrix} \quad (48)$$

解读: $A_{12} = 0.2$ 表示如果当前在盒子 1, 下一次有 0.2 的概率转移到盒子 2。

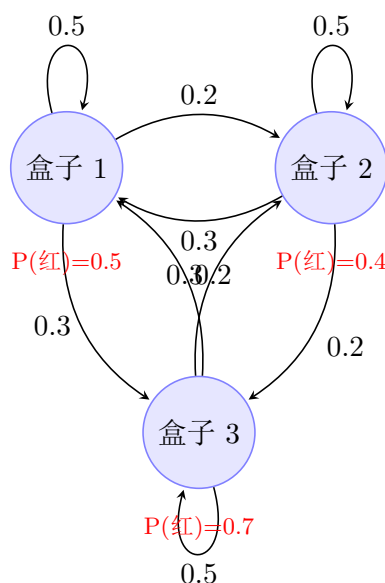
(3) 观测概率矩阵 B (Emission Matrix) 在盒子 j 中摸出某种颜色球的概率 (观测独立性):

$$B = \begin{bmatrix} 0.5 & 0.5 \\ 0.4 & 0.6 \\ 0.7 & 0.3 \end{bmatrix} \quad \begin{array}{l} \text{列 1: 红色} \\ \text{列 2: 白色} \end{array} \quad (49)$$

解读: $B_{21} = 0.4$ 表示在盒子 2 中, 摸到红球的概率是 0.4。

3.8.2 2. 状态转移图 (State Transition Diagram)

上述模型可以用有向图直观表示。节点代表隐状态 (盒子), 边代表转移概率 A_{ij} 。



3.8.3 3. HMM 的三个核心问题推导

假设我们观测到一个序列: $O = (\text{红}, \text{白})$ 。

问题一: 评估 (Evaluation) - 前向算法 问: 这个序列 $O = (\text{红}, \text{白})$ 发生的总概率 $P(O|\lambda)$ 是多少?

解: 我们需要累加所有可能的路径。使用前向变量 $\alpha_t(i)$ 。

第一步 ($t = 1$): 算出第一次摸到“红”且分别在盒子 1,2,3 的概率。

$$\begin{aligned}\alpha_1(1) &= \pi_1 \times B_1(\text{红}) = 0.2 \times 0.5 = 0.10 \\ \alpha_1(2) &= \pi_2 \times B_2(\text{红}) = 0.4 \times 0.4 = 0.16 \\ \alpha_1(3) &= \pi_3 \times B_3(\text{红}) = 0.4 \times 0.7 = 0.28\end{aligned}\tag{50}$$

第二步 ($t = 2$): 从 $t = 1$ 的所有状态转移过来，并摸到“白”。以计算 $\alpha_2(1)$ 为例（即第 2 次在盒子 1 且摸到白球的概率）：

$$\begin{aligned}\alpha_2(1) &= \left[\sum_{i=1}^3 \alpha_1(i) A_{i1} \right] \times B_1(\text{白}) \\ &= (0.10 \times 0.5 + 0.16 \times 0.3 + 0.28 \times 0.2) \times 0.5 \\ &= (0.05 + 0.048 + 0.056) \times 0.5 = 0.154 \times 0.5 = 0.077\end{aligned}\tag{51}$$

同理计算 $\alpha_2(2), \alpha_2(3)$ ，最后求和即为总概率。

问题二：解码 (Decoding) - 维特比算法 问：观测到 (红, 白)，最可能背后的盒子序列是什么？是 $(1 \rightarrow 1)$ 还是 $(3 \rightarrow 2)$ ？

解：我们要找一条“最强路径”。

- 前向算法里我们做的是 **求和 (Sum)**: $\sum \alpha_1(i) A_{ij}$ 。
- 维特比算法里我们做的是 **取最大值 (Max)**: $\max(\delta_1(i) A_{ij})$ 。

在 $t = 2$ 时，我们不看所有路径的总和，只看哪一个上一步状态 i 最容易跳到当前的 j 。我们记录下这个“最强上家”，最后倒着找回去。

问题三：学习 (Learning) - Baum-Welch 算法 问：如果我也不知道矩阵 A, B, π 里的数，只给你一堆红红白白的球的序列，怎么反推这些矩阵？

解：这就是 EM 算法。

- **E 步：**先猜一组 A, B ，算出每一步大概率是在哪个盒子里（计算 $\gamma_t(i)$ 和 $\xi_t(i, j)$ ）。
- **M 步：**根据算出来的概率，“数数”。
 - 比如， A_{12} 的新估计值 = (也就是从 1 跳到 2 的期望次数) / (在 1 的期望总次数)。

4 非参数法 (Non-parametric Methods) (3 学时张燕明)

4.1 1. 引言：为什么要用非参数法？

- 参数法的局限性：
 - 假设样本服从特定的概率分布（如高斯分布），但在实际应用中，数据的分布往往是多峰的、非对称的或未知的。
 - 如果假设的分布模型与真实分布不符，会导致严重的系统误差（偏差大）。
- 非参数法的定义：
 - 不预先假设概率密度函数的数学形式。
 - 完全由数据驱动：概率密度的形式完全由训练样本决定。
- 核心思想：利用样本在特征空间中的局部密度来估计整体概率密度。

4.2 2. 概率密度估计的基本原理

- 概率与密度的关系：设概率密度为 $p(x)$ ，样本落在点 x 附近区域 R 中的概率 P 为：

$$P = \int_R p(x') dx' \quad (52)$$

若区域 R 足够小（体积为 V ），且 $p(x)$ 平滑，则 $P \approx p(x)V$ 。

- 极大似然估计：假设 n 个样本中有 k 个落入区域 R 。根据二项分布，概率 P 的最大似然估计为 $\hat{P} = k/n$ 。
- 通用估计公式：

$$p_n(x) = \frac{k_n/n}{V_n} \quad (\text{样本比例} / \text{体积}) \quad (53)$$

- 收敛性定理 (理论基础)：为了使估计 $p_n(x)$ 收敛于真实密度 $p(x)$ ，当 $n \rightarrow \infty$ 时，必须满足三个条件：

1. $\lim_{n \rightarrow \infty} V_n = 0$ （保证空间分辨率，偏差趋于 0）
2. $\lim_{n \rightarrow \infty} k_n = \infty$ （保证统计稳定性，方差趋于 0）
3. $\lim_{n \rightarrow \infty} k_n/n = 0$ （保证估计收敛，虽然 k 变大，但相对比例要变小）

4.3 3. Parzen 窗估计 (核密度估计)

策略：固定体积 V_n （通常令 $V_n = h_n^d/\sqrt{n}$ 等形式收缩），求落入该体积内的样本数 k_n 。

- 核函数 (Kernel Function) $\varphi(u)$ ：相当于定义了一个“窗口”形状。必须满足：

$$\varphi(u) \geq 0, \quad \int \varphi(u) du = 1$$

- 估计公式:

$$p_n(x) = \frac{1}{n} \sum_{i=1}^n \frac{1}{h_n^d} \varphi\left(\frac{x - x_i}{h_n}\right) \quad (54)$$

直观理解: 在每个样本点 x_i 处堆放一个“小土包”(核函数), 最后把所有土包叠加起来取平均, 就得到了整体的地形(密度函数)。

- 典型窗函数:

- 方窗 (Hypercube Kernel):

$$\varphi(u) = \begin{cases} 1, & |u_j| \leq 1/2 \\ 0, & \text{otherwise} \end{cases}$$

特点: 计算简单, 但估计出的密度函数不连续(阶梯状)。

- 高斯窗 (Gaussian Kernel):

$$\varphi(u) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{1}{2}\|u\|^2\right)$$

特点: 光滑连续, 是应用最广泛的核函数。

- 窗宽 h_n (Bandwidth) 的关键影响:

- h_n 过大: 分辨率低, 细节丢失, 偏差大 (High Bias), 平滑过度。
- h_n 过小: 对噪声敏感, 出现很多尖峰, 方差大 (High Variance), 欠平滑。
- 最佳 h_n : 通常取 $h_n \propto n^{-1/5}$ (在 MSE 准则下)。

- 收敛性分析:

- 均值: $E[p_n(x)]$ 是真实密度 $p(x)$ 与窗函数的卷积(平滑后的版本)。
- 方差: $\text{Var}[p_n(x)] \approx \frac{p(x)}{nV_n}$ 。可见 V_n 不能太小, 否则方差爆炸。

4.4 4. k_n -近邻估计 (k-NN Estimation)

策略: 固定样本数 k_n (通常取 $k_n = \sqrt{n}$), 让体积 V_n 自动适应数据的疏密。

- 方法: 以 x 为中心扩大区域, 直到包含第 k_n 个最近邻, 此时区域半径为 r , 体积为 $V_n = c_d r^d$ 。
- 公式:

$$p_n(x) = \frac{k_n/n}{V_n(x)} \quad (55)$$

- 与 Parzen 窗的对比:

- Parzen 窗在数据稀疏区域效果差(很多窗是空的)。
- k-NN 估计在数据密集处 V_n 小(分辨率高), 稀疏处 V_n 大(平滑好)。

- 缺陷: 估计出的 $p_n(x)$ 拖尾严重, 积分为 ∞ , 不是严格的概率密度函数。

4.5 5. 最近邻分类器 (Nearest Neighbor Rule)

- 几何解释: Voronoi 泰森多边形:

- 将特征空间划分为许多多边形区域, 每个区域包含一个训练样本。
- 区域内任意一点到该样本的距离都小于到其他样本的距离。
- 1-NN 的决策边界就是这些 Voronoi 多胞体的边界。

- 渐进错误率分析: 设 P^* 为贝叶斯最优错误率 (Bayes Error), P 为 1-NN 的错误率。当 $n \rightarrow \infty$ 时:

- 直观逻辑: 当样本无限密时, 测试样本 x 与其最近邻 x' 无限接近, 它们属于某一类的后验概率几乎相同。
- 界限公式:

$$P^* \leq P \leq P^* \left(2 - \frac{c}{c-1} P^* \right) \leq 2P^* \quad (56)$$

其中 c 是类别数。

- 结论: 虽然 1-NN 很简单 (甚至没有训练过程), 但其性能下限非常有保证——错误率绝不会超过贝叶斯错误率的 2 倍。

4.6 6. 计算复杂度的优化

k-NN 的主要缺点是测试阶段计算量大 (Lazy Learning), 需计算与所有 n 个样本的距离。

- 1. 快速搜索 (Fast Search):

- 部分距离法 (Partial Distance): 计算欧氏距离时, 如果前 m 维的累积平方和已经超过了当前的最小距离, 直接在此中断, 不计算剩下的维度。
- 分层搜索树 (KD-Tree):
 - * 将样本空间按坐标轴递归分割, 构成二叉树。
 - * 查询时通过回溯搜索, 平均复杂度从 $O(n)$ 降至 $O(\log n)$ 。
 - * 在高维空间 ($d > 20$) 效果退化为线性扫描。

- 2. 样本集剪辑与压缩:

- 剪辑 (Editing) ——去噪: 利用 Wilson 算法等, 剔除那些被“异类”包围的样本 (通常是噪声或重叠区), 使得决策边界更平滑 (Smooth), 提高泛化能力。
- 压缩 (Condensing) ——减量: 保留定义决策边界的“边界样本”, 剔除那些在 Voronoi 区域内部、对分类不起作用的样本。目的是在保持分类性能不变的前提下, 大幅减少存储和计算量。

4.7 7. 距离度量与切距离

- 闵可夫斯基距离 (Minkowski Metric):

$$L_p(x, y) = (\sum |x_i - y_i|^p)^{1/p}$$

- $p = 1$: 曼哈顿距离。
- $p = 2$: 欧氏距离。
- $p = \infty$: 切比雪夫距离 ($\max |x_i - y_i|$).

- 切距离 (Tangent Distance) - Simard:

- **背景**: 用于手写数字识别。解决图像旋转、缩放、笔画粗细变化导致的欧氏距离失效问题。
- **原理**: 对于一个图像 x , 其经过各种变换 (旋转 α 等) 构成的轨迹是一个流形 (Manifold)。
- 在 x 处做切平面 (Tangent Plane), 用点到切平面的距离代替点到点的距离。
- **效果**: 具有对仿射变换的不变性 (Invariance)。

5 线性分类器设计 (3 学时张燕明)

5.1 引言

- **生成模型 (Generative Models):** 先估计类条件概率密度 $p(x|\omega_i)$ 和先验概率 $P(\omega_i)$, 再利用贝叶斯公式求后验概率。
- **判别模型 (Discriminative Models):** 不估计概率密度, 直接学习后验概率 $P(\omega_i|x)$ 或直接学习判别函数 $g_i(x)$ 的参数。线性分类器属于此类。
- **基本假设:** 假设判别函数 $g(x)$ 具有特定的函数形式 (如线性函数), 利用训练样本集确定函数中的参数。

5.2 线性判别函数与决策面

- 两类问题:

– 定义: $g(x) = w^T x + w_0$ 。其中 w 为权重向量, w_0 为偏置 (阈值)。

– 决策规则:

$$\begin{cases} g(x) > 0 \Rightarrow x \in \omega_1 \\ g(x) < 0 \Rightarrow x \in \omega_2 \\ g(x) = 0 \Rightarrow \text{不定 (决策边界)} \end{cases} \quad (57)$$

– 几何性质:

* 决策面: $H: g(x) = 0$ 是一个超平面, 法向量为 w 。

* 距离: 任意样本 x 到超平面 H 的代数距离为 $r = \frac{g(x)}{\|w\|}$ 。

* 原点距离: 原点到超平面的距离为 $\frac{w_0}{\|w\|}$ 。

- 多类问题:

– 一对其余 (One-vs-All): 构造 c 个两类分类器。存在 “不确定区域”。

– 一对一 (One-vs-One): 构造 $c(c-1)/2$ 个两类分类器, 通过投票决策。也存在不确定区域。

– 线性机器 (Linear Machine): 定义 c 个判别函数 $g_i(x) = w_i^T x + w_{i0}$, 决策规则为取最大值:

$$x \in \omega_i \quad \text{if } g_i(x) > g_j(x), \forall j \neq i$$

这种方法将特征空间划分为 c 个凸区域。

5.3 广义线性判别函数

通过非线性映射 $\varphi(x)$ 将低维线性不可分数据映射到高维空间, 使其在高维空间线性可分。

- **齐次增广表示**：为了简化计算，通常将偏置 w_0 纳入权重向量。

$$y = [1, x_1, \dots, x_d]^T, \quad a = [w_0, w_1, \dots, w_d]^T \Rightarrow g(x) = a^T y \quad (58)$$

此时决策面必然通过增广空间的原点。

- **二次判别函数**： $g(x) = \sum w_{ii}x_i^2 + \sum \sum w_{ij}x_i x_j + \sum w_i x_i + w_0$ 。通过映射 $y = [1, x_1, \dots, x_d, x_1^2, x_1 x_2, \dots]^T$ ，可将其转化为线性形式 $a^T y$ 。
- **维数灾难**：随着阶数增加，映射后的维数呈指数级爆炸，计算复杂度急剧上升。

5.4 感知准则函数 (Perceptron)

假设样本集线性可分。为了统一处理，对 ω_2 类的样本进行**规范化 (Normalization)**： $y \leftarrow -y$ 。此时目标变为寻找 a ，使得对所有样本 y_i ，均有 $a^T y_i > 0$ 。

- **准则函数**：感知器只关注被**错分**的样本集合 $\mathcal{Y}$ (即满足 $a^T y \leq 0$ 的样本)。

$$J_p(a) = \sum_{y \in \mathcal{Y}} (-a^T y) \quad (59)$$

- **梯度下降**： $\nabla J_p = \sum_{y \in \mathcal{Y}} (-y)$ 。
- **批处理感知器算法 (Batch Perceptron)**:

$$a_{k+1} = a_k + \eta_k \sum_{y \in \mathcal{Y}_k} y \quad (60)$$

- **单样本感知器算法 (Fixed-Increment Single-Sample)**：每次只处理一个错分样本 y^k ：

$$a_{k+1} = a_k + y^k \quad (61)$$

- **收敛性定理**：若样本线性可分，固定增量的感知器算法必在**有限步**内收敛。

5.5 松弛方法 (Relaxation Procedures)

感知器要求严格分类正确 ($a^T y > 0$)，松弛方法引入间隔 b ($a^T y \geq b$)，并使用平方误差来平滑目标函数。

- **准则函数 (SQ Error with Margin)**:

$$J_r(a) = \frac{1}{2} \sum_{y \in \mathcal{Y}} \frac{(a^T y - b)^2}{\|y\|^2} \quad (62)$$

其中 $\mathcal{Y}$ 是满足 $a^T y \leq b$ 的错分样本集。

- **单样本更新规则**:

$$a_{k+1} = a_k + \eta \frac{b - a_k^T y^k}{\|y^k\|^2} y^k \quad (63)$$

其中 $0 < \eta < 2$ 。

- $\eta = 1$ ：一步将 a 投影到满足 $a^T y^k = b$ 的超平面上。
- $\eta < 1$ ：欠松弛 (Under-relaxation)。
- $\eta > 1$ ：超松弛 (Over-relaxation)。

5.6 线性最小二乘方法 (MSE)

将不等式求解 $a^T y_i > 0$ 转化为等式求解 $a^T y_i = b_i$ 。定义矩阵 $Y \in \mathbb{R}^{n \times (d+1)}$ ，目标是解方程组 $Ya = b$ 。

- **MSE 准则函数：**

$$J_s(a) = \|Ya - b\|^2 = \sum_{i=1}^n (a^T y_i - b_i)^2 \quad (64)$$

- **伪逆解 (Pseudo-inverse Solution)：** 令 $\nabla J_s = 0$ ，得解析解：

$$a = (Y^T Y)^{-1} Y^T b = Y^\dagger b \quad (65)$$

- **LMS 算法 (Widrow-Hoff)：** 为了避免计算矩阵的逆，使用随机梯度下降 (SGD)：

$$a_{k+1} = a_k + \eta_k (b_k - a_k^T y^k) y^k \quad (66)$$

注意：MSE 追求均方误差最小，其解不一定位于线性可分区域内（对离群点敏感）。

Ho-Kashyap 算法： MSE 方法中 b 是人为固定的（如全 1），这限制了灵活性。Ho-Kashyap 算法同时优化 a 和 b （要求 $b > 0$ ）。

1. 目标：最小化 $J_s(a, b) = \|Ya - b\|^2$ ，约束 $b > 0$ 。
2. 迭代步骤：
 - 固定 b ，更新 a ： $a = Y^\dagger b$
 - 固定 a ，更新 b ： $b_{k+1} = b_k + 2\eta(e_k + |e_k|)$ ，其中 $e_k = Ya - b$ 。
3. 特点：如果问题线性可分，该算法保证收敛；如果不可分，会给出指示（误差 e 不为零但无法继续下降）。

5.7 多类线性判别函数

- **MSE 的多类扩展：** 定义 $W \in \mathbb{R}^{(d+1) \times c}$ ，利用 One-hot 编码将标签转化为目标向量 $z_i \in \{0, 1\}^c$ 。目标函数： $\min_W \sum \|W^T x_i - z_i\|^2$ 。
- **Kesler 构造法：** 将多类分类问题转化为一个高维空间中的两类分类问题。对于属于 ω_1 类的样本 x ，构造 $c - 1$ 个增广样本，使得多类判别条件 $w_1^T x > w_j^T x$ 转化为 $a^T \eta_{1j} > 0$ 。
- **逻辑回归 (Logistic Regression)：** 虽然名字叫回归，实为分类。利用 Sigmoid 函数 $\sigma(z) = \frac{1}{1+e^{-z}}$ 输出后验概率。

$$P(\omega_1|x) = \frac{1}{1 + \exp(-a^T x)} \quad (67)$$

Softmax 回归（多类情形）：

$$P(\omega_i|x) = \frac{\exp(w_i^T x)}{\sum_{j=1}^c \exp(w_j^T x)} \quad (68)$$

通常使用最大似然估计 (MLE) 和梯度上升法进行训练。

Algorithm 1 批处理感知器算法 (Batch Perceptron)

Input: 规范化增广样本集 $Y = \{y_1, \dots, y_n\}$, 学习率 η **Output:** 权向量 a

```

1: 初始化  $a_0, k \leftarrow 0$ 
2: repeat
3:    $Y_k \leftarrow \emptyset$  ▷ 初始化错分样本集
4:   for  $i = 1 \rightarrow n$  do
5:     if  $a_k^T y_i \leq 0$  then
6:        $Y_k \leftarrow Y_k \cup \{y_i\}$ 
7:     end if
8:   end for
9:    $a_{k+1} \leftarrow a_k + \eta \sum_{y \in Y_k} y$  ▷ 梯度下降更新
10:   $k \leftarrow k + 1$ 
11: until  $Y_k = \emptyset$ 
12: return  $a_k$ 

```

6 神经网络和深度学习 (9 学时张燕明)

6.1 人工神经网络发展历程

- 第一阶段：萌芽期 (1940s - 1960s)

- 1943 年, McCulloch 和 Pitts 提出 MP 模型, 是首个神经元数学模型, 利用阈值逻辑模拟神经冲动。
- 1949 年, Hebb 提出 Hebb 学习规则 (“Fire together, wire together”).
- 1958 年, Rosenblatt 提出感知器 (Perceptron), 掀起第一次高潮, 能解决线性分类问题。
- 1969 年, Minsky 和 Papert 出版《Perceptrons》, 证明感知器无法解决异或 (XOR) 等线性不可分问题, 导致神经网络研究进入第一个寒冬。

- 第二阶段：复兴期 (1980s - 1990s)

- 1982 年, Hopfield 提出 Hopfield 网络, 引入能量函数, 不仅具备联想记忆功能, 还解决了 TSP 等优化问题。
- 1986 年, Rumelhart, Hinton 等人重新提出误差反向传播 (BP) 算法, 有效解决了多层网络的训练问题, 掀起第二次高潮。
- 1990 年代中期, 随着 SVM (支持向量机) 等统计学习方法的兴起 (数学理论更完备), 神经网络研究再次进入低谷。

- 第三阶段：爆发期 (Deep Learning, 2006s - 至今)

- 2006 年, Hinton 提出深度置信网络 (DBN) 和 “逐层预训练 + 微调” 的策略, 开启了深度学习时代。
- 2012 年, AlexNet 在 ImageNet 图像分类竞赛中以显著优势夺冠, 深度学习在计算机视觉领域全面爆发。
- 此后, CNN、RNN、Transformer、GAN、GNN 等模型层出不穷, 在语音、NLP、生成模型等领域取得突破。

6.2 人工神经网络基础

- MP 神经元模型:

$$y = f\left(\sum_{i=1}^n w_i x_i - \theta\right) = f(w^T x + b)$$

其中 x 是输入, w 是连接权重, θ 是阈值 (偏置 $b = -\theta$), $f(\cdot)$ 是激活函数。

- 激活函数 (Activation Functions):

- **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$, 将输出压缩到 $(0, 1)$, 可解释为概率。缺点: 易饱和导致梯度消失, 非零中心化。

- **Tanh**: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, 输出范围 $(-1, 1)$, 零中心化, 但仍有梯度消失问题。
- **ReLU (Rectified Linear Unit)**: $f(x) = \max(0, x)$ 。优点: 计算简单, 正区间无梯度消失, 加速收敛。缺点: Dead ReLU 问题 (负区间梯度为 0)。
- **Leaky ReLU / PReLU**: 在负区间引入小斜率, 解决 Dead ReLU 问题。

6.3 单层前馈神经网络 (Perceptron)

- 定义: 只有输入层和输出层, 没有隐含层。
- 几何意义: 在输入空间定义一个超平面 $w^T x + b = 0$ 将数据分为两类。
- 感知器收敛定理: 若训练数据是线性可分的, 则感知器学习算法必在有限步内收敛于一个可行解。若不可分, 则会震荡。
- 局限性: 无法解决 XOR 问题 (线性不可分)。

6.4 多层感知器与误差反向传播算法 (MLP & BP)

- 万有逼近定理 (Universal Approximation): 只要有足够多的隐含层神经元, 单隐层的前馈神经网络可以以任意精度逼近任意连续函数。
- BP 算法推导: 基于链式法则 (Chain Rule)。
 1. 前向传播: 信号从输入层经隐含层传向输出层, 计算误差 E 。
 2. 反向传播: 将误差 E 对权重的梯度 $\frac{\partial E}{\partial w}$ 从输出层向输入层反向传播。
 3. 权重更新: 沿梯度负方向调整权重 $\Delta w = -\eta \frac{\partial E}{\partial w}$ 。
- 误差项 δ :
 - 输出层节点 j : $\delta_j = (d_j - y_j)f'(net_j)$
 - 隐层节点 i : $\delta_i = f'(net_i) \sum_k \delta_k w_{ki}$ (利用后一层的误差加权求和)。

6.5 BP 算法讨论与优化

- 学习模式:
 - 批量学习 (Batch Learning): 利用所有样本计算总梯度后更新。方向准确, 但计算量大, 易陷入局部极小。
 - 随机梯度下降 (SGD): 每输入一个样本就更新一次。计算快, 利于跳出局部极小, 但震荡剧烈。
 - 小批量学习 (Mini-batch): 折中方案, 如 Batch size=32, 64。深度学习中最常用。
- 常见问题与对策:
 - 局部极小值 (Local Minima):

- * 引入动量项 (Momentum): $\Delta w(t) = -\eta \nabla E + \alpha \Delta w(t-1)$, 利用惯性加速收敛并冲过浅坑。
- * 随机初始化权重、添加随机噪声。
- 平坦区 (Plateau): 梯度接近 0, 收敛慢。对策: 使用自适应学习率算法 (如 AdaGrad, RMSProp, Adam)。
- 过拟合 (Overfitting):
 - * 早停 (Early Stopping): 验证集误差不再下降时停止。
 - * 正则化 (Weight Decay): 在损失函数加入 L_2 范数 $\lambda \|w\|^2$ 。
 - * Dropout: 训练时随机“关闭”一部分神经元, 测试时全开 (相当于集成学习)。
 - * 数据增强 (Data Augmentation): 旋转、裁剪、加噪等。
- 梯度消失/爆炸:
 - * 使用 ReLU 激活函数。
 - * Batch Normalization (BN): 对每层输入做归一化, 加速训练, 缓解梯度问题。
 - * ResNet 残差连接。

6.6 反馈神经网络 (Hopfield Network)

- 特点: 网络中存在反馈回路, 是一个动力学系统。系统状态随时间演化, 最终稳定在吸引子 (Attractor)。
- 能量函数 (Energy Function):

$$E = -\frac{1}{2} \sum_{i \neq j} w_{ij} x_i x_j - \sum_i \theta_i x_i$$

网络演化过程中能量单调递减 ($dE/dt \leq 0$), 最终达到极小值 (稳定状态)。

- 应用:
 - 联想记忆 (Associative Memory): 部分/噪声输入能恢复出完整的记忆模式 (内容寻址)。
 - 优化计算: 如求解 TSP 旅行商问题 (将目标函数映射为网络能量函数)。

6.7 径向基函数网络 (RBF)

- Cover 定理: 将低维空间的非线性可分模式映射到高维空间, 它们变成线性可分的概率将趋于 1。
- 结构: 输入层 $\rightarrow$ 隐层 (非线性映射) $\rightarrow$ 输出层 (线性组合)。
- 径向基函数: 隐层采用高斯函数 $\phi(x) = \exp(-\frac{\|x-c\|^2}{2\sigma^2})$, 具有局部响应特性。
- 训练策略:

1. 无监督学习确定中心 c_i (如 K-Means 聚类)。
 2. 监督学习确定输出层权重 w_i (即线性回归, 有解析解)。
- **正则化网络:** 在 RBF 损失函数中引入平滑项, 即 Tikhonov 正则化。

6.8 自组织映射 (SOM)

- **原理:** 基于大脑皮层的侧抑制现象, 是一种无监督竞争式学习。
- **目标:** 将高维输入空间映射到低维 (通常 2D) 离散网格, 且保持拓扑结构不变 (相似的输入映射到邻近的神经元)。
- **算法流程:**
 1. 竞争: 找到与输入 x 距离最近的获胜神经元 (Winner)。
 2. 合作: 定义获胜神经元的邻域, 邻域内的神经元也被激活。
 3. 更新: $w_j(t+1) = w_j(t) + \eta(t)h_{j,win}(t)(x - w_j(t))$, 拉动权重向量向输入靠近。
- **应用:** 特征降维、聚类、数据可视化。

6.9 深度学习简介

- **浅层 vs 深层:**
 - 浅层模型 (如 SVM, 单隐层 MLP): 人工设计特征 + 浅层分类器。
 - 深层模型: 自动学习特征 (Representation Learning), 实现 “端到端 (End-to-End)” 学习。
- **核心思想:** 通过多层非线性变换, 从低级特征 (边缘) 逐层抽象出高级特征 (部件、物体)。
- **历史转折:** Hinton 在 2006 年提出的 “贪婪逐层预训练 (Greedy Layer-wise Pre-training)” 解决了深层网络难以训练的问题, 尽管后来随着 ReLU 和 BN 的出现, 预训练不再是必须的。

6.10 波尔兹曼机 (Boltzmann Machine)

- **BM:** 一种随机神经网络, 神经元状态以概率更新, 基于能量模型。
- **受限波尔兹曼机 (RBM):**
 - 结构: 层内无连接, 层间全连接 (二部图结构)。
 - 训练: 对比散度 (Contrastive Divergence, CD-k) 算法。通过 Gibbs 采样近似梯度的计算。
- **深度置信网络 (DBN):** 由多个 RBM 堆叠而成。
- **深度波尔兹曼机 (DBM):** 层间无向连接的深层模型。

6.11 自编码器 (Autoencoder)

- **基本原理:** $Encoder: z = f(x)$, $Decoder: \hat{x} = g(z)$ 。目标是 minimize 重构误差 $\|x - \hat{x}\|^2$ 。
- **作用:** 降维 (类似非线性 PCA)、特征提取、生成模型。
- **变体:**
 - 降噪自编码器 (Denoising AE): 输入加噪声 $\tilde{x}$, 强迫网络恢复原始 x , 学习鲁棒特征。
 - 稀疏自编码器 (Sparse AE): 限制隐层神经元的激活率 (大部分时间不激活)。
 - 堆叠自编码器 (Stacked AE): 逐层堆叠进行预训练。

6.12 卷积神经网络 (CNN)

专为处理网格数据 (如图像) 设计。

- **核心组件:**
 - 卷积层 (Conv): 利用卷积核提取局部特征。特性: 局部连接 (Local Connectivity) 和 权值共享 (Weight Sharing), 大幅减少参数量。
 - 池化层 (Pooling): 下采样 (Max Pooling / Avg Pooling)。作用: 减少特征图尺寸, 增大感受野, 提供平移不变性。
 - 全连接层 (FC): 用于最终的分类决策。
- **经典架构:**
 - LeNet-5 (1998): 早期 CNN, 用于手写数字识别。
 - AlexNet (2012): 深度学习爆发的标志, 引入 ReLU, Dropout, GPU 加速。
 - VGG (2014): 使用连续的 3×3 小卷积核代替大核, 加深网络。
 - GoogLeNet (Inception): 引入 Inception 模块, 多尺度卷积并行。
 - ResNet (2015): 引入残差连接 (Skip Connection), 解决了深层网络 (如 152 层) 的梯度消失/退化问题。

6.13 循环神经网络 (RNN) 与 LSTM

专为序列数据 (语音、文本) 设计。

- **RNN 基本结构:** 引入隐状态 h_t , 当前输出不仅依赖当前输入 x_t , 还依赖上一时刻状态 h_{t-1} 。

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

- **BPTT (Backprop Through Time):** 按时间展开进行反向传播。
- **问题:** 长序列训练面临梯度消失或梯度爆炸, 难以捕捉长距离依赖。
- **LSTM (Long Short-Term Memory):** 引入细胞状态 (Cell State) C_t 和三个门控单元:

- 遗忘门 (Forget Gate) f_t : 决定丢弃多少旧信息。
- 输入门 (Input Gate) i_t : 决定更新多少新信息。
- 输出门 (Output Gate) o_t : 决定输出多少当前状态。
- **GRU (Gated Recurrent Unit)**: LSTM 的简化版, 合并了细胞状态和隐状态。

6.14 注意力机制与 Transformer

- **Seq2Seq 模型**: Encoder-Decoder 架构, 早期使用 RNN 处理机器翻译。存在定长向量瓶颈。
- **注意力机制 (Attention Mechanism)**: 解码时不再只看最后一个隐状态, 而是通过“注意力权重”动态加权 Encoder 的所有输出, 聚焦于输入序列的相关部分。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Transformer (2017)**: 完全抛弃 RNN/CNN, 纯基于 Attention 的架构。
 - Self-Attention (自注意力): 捕捉序列内部单词间的关联。
 - Multi-Head Attention: 在不同子空间并行关注不同信息。
 - Positional Encoding: 由于没有 RNN 的时序结构, 需手动注入位置信息。
- **BERT (Bi-directional Encoder Representations from Transformers)**: 基于 Transformer Encoder 的预训练模型。通过 Masked LM 和 Next Sentence Prediction 任务在大规模语料上预训练, 再在下游任务微调, 刷新了 NLP 各项记录。

6.15 图神经网络 (GNN)

- **动机**: 处理非欧几里得空间数据 (如社交网络、分子结构、知识图谱)。
- **核心机制: 消息传递 (Message Passing)**。节点的特征更新依赖于其邻居节点特征的聚合。

$$h_v^{(l+1)} = \sigma\left(W \cdot \text{AGGREGATE}(\{h_u^{(l)}, \forall u \in \mathcal{N}(v)\})\right)$$

- **常见模型**:
 - GCN (Graph Convolutional Network): 基于谱图理论或空间邻域聚合。
 - GAT (Graph Attention Network): 在聚合邻居信息时引入注意力机制, 给不同邻居分配不同权重。

7 特征提取与选择 (6 学时张燕明)

7.1 维数灾难 (Curse of Dimensionality)

- 现象：随着特征维数的增加，计算量呈指数级增长；在样本数量固定的情况下，分类器的性能通常先由差变好，达到峰值后，随着维数继续增加反而变差（Hughes 现象）。
- 原因：
 - 数据稀疏性：高维空间体积急剧膨胀，样本密度变得极低，为了维持同样的密度，所需样本量呈指数级增加。
 - 距离失效：在高维空间中，几乎所有点对之间的距离都趋于相等，欧氏距离等度量失效。
- 对策：降维（特征提取与特征选择）。

7.2 特征提取 (Feature Extraction)

- 定义：通过某种数学变换，将原始高维特征空间映射到一个低维子空间，从而生成一组新的特征（原始特征的组合）。
- 目的：降低维度、去除噪声、提高计算效率、可视化。
- 分类：
 - 无监督：仅利用数据分布信息（如 PCA）。
 - 有监督：利用类别标签信息（如 LDA）。

7.3 线性方法

7.3.1 1. 主成分分析 (PCA)

- 思想：寻找一个正交投影方向，使得数据投影后的 ** 方差最大 **（保留最多信息）或 ** 重构误差最小 **。
- 求解：对样本协方差矩阵 S 进行特征值分解。

$$S = \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^T \quad (69)$$

$$Su_i = \lambda_i u_i \quad (70)$$

选取最大的 d' 个特征值对应的特征向量组成投影矩阵。

- 特点：无监督，线性变换，去相关。

7.3.2 2. 线性判别分析 (Fisher LDA)

- **思想**: 寻找一个投影方向, 使得投影后 ** 类内散度 (S_w) 最小 **, 同时 ** 类间散度 (S_b) 最大 ** (Fisher 准则)。

- **优化目标**:

$$J(w) = \frac{w^T S_b w}{w^T S_w w} \quad (71)$$

- **求解**: 转化为广义特征值问题 $S_b w = \lambda S_w w$ 。
- **特点**: 有监督, 通过最大化类间距离和最小化类内距离来增强分类性能。受限于类别数 c , 最多提取 $c - 1$ 个特征。

7.3.3 3. 多维缩放 (MDS)

- **思想**: 在低维空间中保持原始高维空间样本间的距离 (或相似度) 不变。

7.4 非线性方法 (流形学习)

针对位于非线性流形上的数据, 线性方法失效。

- **核方法 (Kernel Methods)**: 如 Kernel PCA (KPCA)、Kernel LDA (KDA)。利用核技巧将线性方法扩展到非线性空间。
- **等距映射 (ISOMAP)**: 基于 MDS, 但使用 ** 测地线距离 (Geodesic Distance) ** 代替欧氏距离。通过构建近邻图并计算最短路径来近似测地线距离, 保持数据的全局几何结构。
- **局部线性嵌入 (LLE)**:
 - 假设数据在局部是线性的。
 - **步骤**: 1. 寻找近邻; 2. 用近邻线性重构中心点, 求重构权重; 3. 在低维空间保持重构权重不变, 求解低维坐标。
 - **特点**: 保持数据的局部邻域关系。
- **拉普拉斯特征映射 (Laplacian Eigenmaps)**: 基于图论, 通过图拉普拉斯算子的特征分解, 使在高维空间相近的点在低维空间也尽可能相近。
- **t-SNE**: 主要用于高维数据可视化, 通过概率分布匹配保持局部结构。

7.5 特征选择 (Feature Selection)

- **定义**: 从原始特征集合中挑选出最具代表性的子集, 不改变特征的原始物理意义。
- **搜索策略**:
 - **完全搜索**: 如分支限界法 (Branch and Bound), 保证找到最优解, 但计算量大, 仅适用于单调性准则。

– 启发式搜索：

- * 序列前向选择 (SFS)：从空集开始，每次加入一个效果最好的特征。
- * 序列后向选择 (SBS)：从全集开始，每次剔除一个最无用的特征。
- * 增 L 减 R 法 (L-R Selection)：结合前向和后向，避免陷入局部极小。

• 评价准则与方法分类：

过滤式 (Filter)：特征选择独立于后续分类器。使用距离度量、信息增益、相关系数等统计指标评价特征重要性。速度快，通用性强，但精度可能不如包裹式。

包裹式 (Wrapper)：直接使用后续分类器的性能（如验证集准确率）作为评价准则。精度高，但计算开销大，易过拟合。

嵌入式 (Embedded)：特征选择与模型训练过程融合。如 LASSO (L_1 正则化) 导致权重稀疏，或决策树/随机森林中的特征重要性评分。

8 模型选择 (3 学时张燕明)

8.1 模型选择原则

- 奥卡姆剃刀原理 (Occam's Razor):
 - 核心思想: “如无必要, 勿增实体”。在性能相近的模型中, 优先选择更简单 (参数更少、结构更精简) 的模型。
 - 应用: 正则化 (Regularization), 即在损失函数中增加惩罚项 (如 L_1, L_2 范数) 以限制模型复杂度。
- 没有免费的午餐定理 (No Free Lunch Theorem, NFL):
 - 结论: 不存在一个在所有可能问题上都最优的算法。算法的性能是与具体问题相关的。
 - 推论: 脱离具体问题谈论 “最好的算法” 没有意义, 必须结合数据的先验知识和分布特性来选择模型。

8.2 模型评价标准

- 基本概念:
 - 训练误差 (Training Error): 模型在训练集上的误差, 易受过拟合影响。
 - 泛化误差 (Generalization Error): 模型在未见过的测试数据上的误差, 是评价模型真实性能的关键指标。
- 样本划分与验证方法:
 - 留出法 (Holdout): 将数据随机划分为训练集和测试集 (如 7:3 或 8:2)。
 - K 折交叉验证 (K-fold Cross Validation): 将数据均分为 K 份, 轮流用其中 1 份做测试, 其余 $K - 1$ 份做训练, 取 K 次结果的平均值。常用于超参数选择。
 - 留一法 (Leave-One-Out, LOO): $K = N$ 的特例, 每次只留一个样本做测试。方差大, 计算代价高。
 - 自助法 (Bootstrap): 通过有放回采样生成训练集。约 36.8% 的样本未被选中 (Out-of-Bag), 可用于测试。

8.3 基于偏差和方差的分类器设计准则

- 误差分解: 泛化误差 = 偏差 (Bias)² + 方差 (Variance) + 噪声 (Noise)。
- 偏差-方差权衡 (Bias-Variance Trade-off):
 - 高偏差 (High Bias): 模型过于简单 (欠拟合), 无法捕捉数据规律 (如用直线拟合曲线)。
 - 高方差 (High Variance): 模型过于复杂 (过拟合), 对训练数据的微小扰动非常敏感。

- **设计准则：**随着模型复杂度增加，偏差降低，方差升高。最优模型应位于两者之和最小的平衡点。

图 4: 模型复杂度与误差的关系：训练误差单调下降，测试误差呈 U 型曲线

8.4 分类器集成 (Ensemble Learning)

- **核心思想：**“三个臭皮匠，顶个诸葛亮”。通过组合多个基分类器 (Base Classifiers) 来提升整体性能。
- **有效条件：**基分类器必须具有**差异性 (Diversity)**，且精度优于随机猜测 (Error Rate < 0.5)。
- **常用方法：**

1. Bagging (Bootstrap Aggregating):

- 并行训练多个基分类器，每个使用不同的 Bootstrap 样本集。
- 最终决策：分类任务投票 (Voting)，回归任务平均。
- 代表：随机森林 (Random Forest)。主要降低**方差**。

2. Boosting (提升方法):

- 串行训练，后续分类器重点关注前序分类器分错的样本。
- 代表：Adaboost, GBDT, XGBoost。主要降低**偏差**。

3. Stacking (层叠泛化):

- 将初级分类器的输出作为次级分类器（元学习器）的输入。

8.5 Adaboost 学习方法及其应用

Adaboost (Adaptive Boosting) 是一种前向分步加法模型。

8.5.1 1. Adaboost 算法流程

输入：训练集 $T = \{(x_1, y_1), \dots, (x_N, y_N)\}, y_i \in \{-1, +1\}$ 。

1. **初始化权重：** $w_{1,i} = 1/N, \quad i = 1, \dots, N$ 。
2. **循环 $m = 1$ to M ：**
 - 基于当前权重分布 D_m 训练弱分类器 $G_m(x)$ 。
 - 计算加权分类错误率： $e_m = \sum_{i=1}^N w_{m,i} I(G_m(x_i) \neq y_i)$ 。
 - 计算弱分类器权重（置信度）：

$$\alpha_m = \frac{1}{2} \ln \frac{1 - e_m}{e_m} \quad (72)$$

- 更新样本权重:

$$w_{m+1,i} = \frac{w_{m,i}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)) \quad (73)$$

(被错分的样本权重变大, 被正确分类的样本权重变小)。

3. 输出强分类器:

$$F(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right) \quad (74)$$

8.5.2 2. 理论性质

Adaboost 的训练误差随弱分类器数量 M 的增加呈指数级下降:

$$\text{Training Error} \leq \exp(-2 \sum_{m=1}^M \gamma_m^2)$$

其中 $\gamma_m = 0.5 - e_m$ 。

8.5.3 3. 应用: Viola-Jones 人脸检测

- **Haar 特征:** 利用矩形区域的像素和差值描述边缘、纹理特征。
- **积分图 (Integral Image):** 一种加速算法, 使得任意矩形区域的像素和计算复杂度为 $O(1)$ 。
- **级联结构 (Cascade):**
 - 将多个强分类器串联。
 - **早期拒绝:** 前面的分类器较简单 (特征少), 阈值宽松, 旨在快速排除大量明显的非人脸窗口 (负样本)。
 - **后期精细:** 后面的分类器较复杂, 负责区分难分辨的样本。
 - 极大地提高了检测速度。

9 聚类分析 (6 学时张燕明)

9.1 基本概念

- **定义：**无监督学习过程，将样本集 X 划分为 K 个互不相交的子集（簇） $D_1, \dots, D_K$ ，使得同一簇内样本相似度高，不同簇间样本相似度低。
- **距离度量：**
 - Minkowski 距离 (L_q norm): $d(x, y) = (\sum |x_i - y_i|^q)^{1/q}$ 。($q = 2$ 为欧氏距离, $q = 1$ 为曼哈顿距离)。
 - 马氏距离 (Mahalanobis): $d(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$ ，考虑了特征间的相关性。
 - 余弦相似度 (Cosine): $s(x, y) = \frac{x^T y}{\|x\| \|y\|}$ ，主要关注方向而非大小。

9.2 K 均值聚类 (K-Means)

- **目标函数：**最小化簇内平方误差和 (SSE)。

$$J = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \|x_n - \mu_k\|^2 \quad (75)$$

其中 $z_{nk} \in \{0, 1\}$ 是指示变量，表示样本 n 是否属于簇 k 。

- **算法流程 (Lloyd's Algorithm):**

1. **初始化：**随机选择 K 个中心 $\mu_1, \dots, \mu_K$ 。
2. **分配 (Assignment):** 固定 μ ，更新 z 。将每个样本分配到最近的中心。

$$z_{nk} = 1 \iff k = \arg \min_j \|x_n - \mu_j\|^2$$

3. **更新 (Update):** 固定 z ，更新 μ 。计算每个簇的新均值。

$$\mu_k = \frac{\sum_n z_{nk} x_n}{\sum_n z_{nk}}$$

4. **迭代：**重复步骤 2-3 直至收敛（中心不再变化或目标函数下降小于阈值）。

- **K-Means++ 初始化：**为了解决对初始值敏感的问题，K-Means++ 按照与已有中心距离的平方成正比的概率选择下一个中心，尽可能使初始中心分散。
- **K 值选择：肘部法则 (Elbow Method/Knee Finding):** 绘制目标函数 J 随 K 变化的曲线，选择曲线出现“拐点”处的 K 值。

9.3 模糊 K 均值聚类 (GMM & EM)

传统 K-Means 是硬分配 (Hard Assignment)，即样本只能属于一个簇。PPT 中重点介绍了基于概率的 ** 高斯混合模型 (GMM) ** 作为软聚类 (Soft Assignment) 方法。

- **高斯混合模型 (GMM)**: 假设数据由 K 个高斯分布混合生成:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k) \quad (76)$$

其中 π_k 是混合系数 (先验概率), $\sum \pi_k = 1$ 。

- **EM 算法 (Expectation-Maximization)**: 用于求解含有隐变量 (簇标签) 的极大似然估计问题。

- **E 步 (Expectation)**: 计算样本属于第 k 个簇的后验概率 (Responsibility)。

$$\gamma_k(x_n) = \frac{\pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)} \quad (77)$$

- **M 步 (Maximization)**: 利用加权样本更新参数。

$$\mu_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma_k(x_n) x_n, \quad \Sigma_k^{new} = \frac{1}{N_k} \sum_{n=1}^N \gamma_k(x_n) (x_n - \mu_k)(x_n - \mu_k)^T$$

其中 $N_k = \sum_{n=1}^N \gamma_k(x_n)$ 。

- **与 K-Means 关系**: 当 GMM 的协方差矩阵为 $\sigma^2 I$ 且 $\sigma^2 \rightarrow 0$ 时, GMM 退化为 K-Means。

9.4 层次聚类 (Hierarchical Clustering)

- **策略**:

- **凝聚 (Agglomerative)**: 自底向上, 开始时每个样本是一类, 每次合并距离最近的两个类。
- **分裂 (Divisive)**: 自顶向下, 开始时所有样本是一类, 每次分裂差异最大的类。

- **簇间距离度量 (Linkage)**:

- **最短距离 (Single Linkage)**: $d(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$ 。易产生“链式效应”。
- **最长距离 (Complete Linkage)**: $d(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$ 。倾向于产生球状簇。
- **平均距离 (Average Linkage)**: 所有点对距离的平均值。

- **可视化**: 使用树状图 (Dendrogram) 展示聚类过程, 横轴代表样本, 纵轴代表合并时的距离 (相似度)。

9.5 谱聚类 (Spectral Clustering)

基于图论的聚类方法, 能够发现非凸形状的簇。

- **基本步骤**:

1. **构图**: 构建样本的相似度图 (如 k-近邻图), 得到邻接矩阵 W 和度矩阵 D (对角阵, 元素为节点度数)。
 2. **拉普拉斯矩阵**: 计算图拉普拉斯矩阵。
 - 未归一化: $L = D - W$
 - 对称归一化: $L_{sym} = I - D^{-1/2}WD^{-1/2}$
 - 随机游走归一化: $L_{rw} = I - D^{-1}W$
 3. **特征分解**: 计算 L 的前 k 个最小特征值对应的特征向量 $u_1, \dots, u_k$ 。
 4. **新特征空间**: 将特征向量作为列组成矩阵 $U \in \mathbb{R}^{N \times k}$, 将 U 的每一行视为一个新的 k 维样本 y_i 。
 5. **聚类**: 对 y_i 进行 K-Means 聚类得到最终结果。
- **原理**: 谱聚类本质上是图切割问题 (如 RatioCut 或 Ncut) 的松弛求解。最小化切图代价相当于让簇内连接紧密, 簇间连接稀疏。

9.6 在线聚类与竞争学习

- **竞争学习 (Competitive Learning)**: 一种在线 (Online) 聚类方法, 样本按顺序输入, 每次只更新获胜的那个原型 (Winner-take-all)。

$$\Delta w_{winner} = \eta(x - w_{winner})$$

- **BSAS (Basic Sequential Algorithmic Scheme)**:
 - 按顺序处理每个样本。
 - 计算样本与现有簇中心的距离。
 - 若最小距离大于阈值 θ , 则生成新簇; 否则归入最近簇并更新中心。
 - 缺点: 结果依赖于样本输入顺序。

9.7 集成聚类 (Ensemble Clustering)

- **动机**: 单一聚类算法对参数 (如 K 值、初始化) 敏感。集成聚类通过组合多个基聚类结果 (Base Partitions) 来提高聚类的**稳健性 (Robustness)** 和 **稳定性**。
- **多样性构建**:
 - 使用不同的聚类算法 (如 K-Means + 层次 + 谱聚类)。
 - 使用不同的参数 (如不同的 K, 不同的初始化)。
 - 数据重采样 (Bagging, Random Projection)。
- **一致性函数 (Consensus Function)**: 将多个基聚类结果合并为一个最终的一致性划分 (Consensus Partition), 常用方法包括基于共联矩阵 (Co-association Matrix) 的投票法或图划分法。

10 支持向量机与核方法 (6 学时向世明)

10.1 统计学习理论基础

- **核心目标:** 在有限样本下寻求结构风险最小化 (**Structural Risk Minimization, SRM**), 即在保证分类精度的同时, 降低模型复杂度, 从而提高泛化能力。
- **VC 维 (VC Dimension):**
 - **定义:** 假设空间 H 能“打散 (Shatter)”的最大样本数。
 - **物理意义:** 衡量函数集的学习能力 (容量)。VC 维越高, 模型越复杂, 越容易过拟合。
 - **线性模型:** d 维空间中线性分类器的 VC 维为 $d + 1$ 。
 - **SVM 策略:** 通过最大化间隔 (Margin), 实际上是在限制函数集的有效 VC 维, 从而获得更好的泛化界。

10.2 线性可分支持向量机 (Hard Margin SVM)

假设训练数据线性可分, 目标是寻找最大间隔超平面。

- **超平面方程:** $w^T x + b = 0$ 。
- **几何间隔:** 两类支持向量到超平面的距离之和, 计算为 $\gamma = \frac{2}{\|w\|}$ 。
- **原始优化问题 (Primal Problem):**

$$\begin{aligned} \min_{w, b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, N \end{aligned} \quad (78)$$

- **支持向量:** 满足 $y_i(w^T x_i + b) = 1$ 的样本点, 它们决定了最终的决策边界。

10.3 线性支持向量机 (Soft Margin SVM)

解决数据线性不可分或存在噪声的情况。

- **松弛变量 (Slack Variables):** 引入 $\xi_i \geq 0$, 允许样本偏离间隔边界。
- **优化问题:**

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned} \quad (79)$$

- **惩罚参数 C :**
 - C 较大: 对误分类惩罚重, 模型趋向于硬间隔 (低偏差, 高方差)。
 - C 较小: 容忍更多误分, 模型趋向于更宽的间隔 (高偏差, 低方差)。
- **合页损失 (Hinge Loss):** SVM 的优化目标等价于 L_2 正则项 + Hinge Loss ($[1 - y_i f(x_i)]_+$)。

10.4 对偶理论与核技巧

10.4.1 对偶问题 (Dual Problem)

利用拉格朗日乘子法，将原始问题转化为对偶问题，便于引入核函数。

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \end{aligned} \quad (80)$$

• KKT 条件:

- $\alpha_i = 0$: 样本被正确分类且远离边界（非支持向量）。
- $0 < \alpha_i < C$: 样本位于间隔边界上（支持向量， $\xi_i = 0$ ）。
- $\alpha_i = C$: 样本位于间隔边界内（可能被误分，支持向量， $\xi_i > 0$ ）。

10.4.2 核技巧 (Kernel Trick)

通过非线性映射 $\phi(x)$ 将低维数据映射到高维特征空间。定义核函数 $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ ，避免显式计算高维映射。

常用核函数:

1. 线性核 (Linear): $K(x, z) = x^T z$ 。适用于特征维数高的数据（如文本分类）。
2. 多项式核 (Polynomial): $K(x, z) = (x^T z + 1)^d$ 。
3. 高斯径向基核 (RBF):

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right) = \exp(-\gamma\|x - z\|^2) \quad (81)$$

对应无穷维特征空间。 γ 越大，模型越复杂（越“尖”），易过拟合。

10.4.3 算法实现: SVM 预测流程

SVM 的预测仅依赖于支持向量（ $\alpha_i^* > 0$ 的样本）。

Algorithm 2 基于核函数的 SVM 预测算法

Input: 训练好的支持向量集 $S = \{(x_i, y_i, \alpha_i^*) \mid \alpha_i^* > 0\}$ ，偏置 b^* ，核函数 $K(\cdot, \cdot)$ ，测试样本 x

Output: 预测类别 $\hat{y}$

- 1: $f \leftarrow 0$
 - 2: **for** 每个支持向量 $(x_i, y_i, \alpha_i^*) \in S$ **do**
 - 3: $f \leftarrow f + \alpha_i^* \cdot y_i \cdot K(x_i, x)$ ▷ 计算加权核函数值
 - 4: **end for**
 - 5: $f \leftarrow f + b^*$
 - 6: $\hat{y} \leftarrow \text{sign}(f)$
 - 7: **return** $\hat{y}$
-

10.5 SVM 的扩展

- **核逻辑回归 (Kernel Logistic Regression, KLR):** 将 LR 的线性部分 $w^T x$ 替换为核形式 $\sum \alpha_i K(x_i, x)$ 。KLR 输出概率，解不具有稀疏性。
- **多分类 SVM:**
 - **One-vs-Rest (OvR):** 训练 k 个分类器，取置信度最高者。
 - **One-vs-One (OvO):** 训练 $k(k-1)/2$ 个分类器，采用投票法。
- **支持向量回归 (SVR):** 引入 ϵ -不敏感损失函数，只惩罚超出 ϵ 管道的误差。

11 决策树方法 (3 学时向世明)

11.1 特征选择准则

决策树生成的关键在于选择最优划分属性。

1. 信息增益 (ID3 算法):

$$g(D, A) = H(D) - H(D|A)$$

其中 $H(D)$ 为经验熵, $H(D|A)$ 为条件熵。缺点是偏向于选择取值较多的特征 (如 ID 号)。

2. 信息增益比 (C4.5 算法):

$$g_R(D, A) = \frac{g(D, A)}{H_A(D)}$$

引入属性 A 的固有熵 $H_A(D)$ 进行惩罚。C4.5 还能处理连续值 (二分法) 和缺失值。

3. 基尼指数 (CART 算法):

$$\text{Gini}(p) = 1 - \sum_{k=1}^K p_k^2$$

反映了从数据集中随机抽取两个样本类别不一致的概率。CART 生成的是二叉树。

11.2 决策树生成算法

典型的决策树生成是一个递归过程。

Algorithm 3 决策树生成算法 (Generic)**Input:** 训练数据集 D , 特征集 A , 阈值 ϵ **Output:** 决策树 T

```

1: function GENERATE TREE( $D, A$ )
2:   if  $D$  中所有实例属于同一类  $C_k$  then
3:     return 单结点树  $T$ , 标记类别为  $C_k$ 
4:   end if
5:   if  $A = \emptyset$  OR  $D$  在  $A$  上取值相同 then
6:     return 单结点树  $T$ , 标记为  $D$  中实例数最大的类
7:   end if
8:   从  $A$  中选择最优划分特征  $A_g$  (基于信息增益/增益比/基尼指数)
9:   if  $A_g$  的优选指标小于  $\epsilon$  then
10:    return 单结点树  $T$ , 标记为  $D$  中实例数最大的类
11:  end if
12:  初始化  $T$  为根结点, 特征为  $A_g$ 
13:  for  $A_g$  的每一个可能取值  $a_i$  do
14:     $D_i \leftarrow D$  中特征  $A_g$  取值为  $a_i$  的子集
15:    if  $D_i$  为空 then
16:      将该分支标记为叶结点, 类别为  $D$  中最多的类
17:    else
18:       $T_{sub} \leftarrow \text{GENERATE TREE}(D_i, A \setminus \{A_g\})$  ▷ 递归生成子树
19:      将  $T_{sub}$  连接为  $T$  的子结点
20:    end if
21:  end for
22:  return  $T$ 
23: end function

```

11.3 决策树剪枝 (Pruning)

为了防止过拟合, 需对树进行简化。

- **预剪枝 (Pre-pruning):** 在生成过程中, 若当前划分不能提升泛化性能 (如验证集精度), 则停止划分。
- **后剪枝 (Post-pruning):** 先生成完整树, 再自底向上考察非叶结点。若替换为叶结点能提升性能, 则剪枝。
- **代价复杂度剪枝 (CCP):** CART 使用的方法。最小化损失函数 $C_\alpha(T) = C(T) + \alpha|T|$ 。

11.4 随机森林 (Random Forests)

随机森林是基于 **Bagging** 的集成学习方法, 引入了样本随机和特征随机。

- **泛化误差**：取决于单棵树的强度 (Strength) 和树之间的相关性 (Correlation)。强度越大、相关性越低，性能越好。
- **OOB (Out-of-Bag) 估计**：利用 Bootstrap 未采样到的样本（约 36.8%）作为验证集，无需交叉验证。

Algorithm 4 随机森林构建算法

Input: 训练集 D , 树的棵数 K , 特征子集大小 m

Output: 随机森林模型 $RF = \{h_1, \dots, h_K\}$

- 1: **for** $k = 1$ to K **do**
 - 2: **Bootstrap 采样**: 从 D 中有放回随机采样 N 次, 得到训练子集 D_k
 - 3: **树生长**: 使用 D_k 训练决策树 h_k
 - 4: 在树的每个节点分裂时:
 - 5: 1. 从所有 M 个特征中**随机**选择 m 个特征 ($m \ll M$)
 - 6: 2. 在这 m 个特征中计算最优分裂点
 - 7: 3. 树生长到最大深度, **不进行剪枝**
 - 8: $RF \leftarrow RF \cup \{h_k\}$
 - 9: **end for**
 - 10: **预测**: 对于新样本 x , 分类任务采用**多数投票**, 回归任务采用**平均值**。
-

12 模式识别前沿趋势 (3 学时向世明)

12.1 课程内容总体回顾

12.2 模式识别发展现状与新动态

12.3 模式识别中的挑战性问题

13 考核 (6 学时向世明)

13.1 考试 + 答疑

附录：模式识别常用 LaTeX 模板

1. 学术三线表模板

表 1: 不同分类器在 Iris 数据集上的错误率对比

算法	训练集错误率	测试集错误率	计算耗时 (ms)
KNN ($K = 3$)	0.02	0.04	12
Fisher LDA	0.05	0.06	2
SVM (RBF Kernel)	0.01	0.03	45

2. 图片插入模板 (已注释)

3. 模式识别核心公式模板

贝叶斯决策规则： 后验概率计算：

$$P(\omega_i|\mathbf{x}) = \frac{p(\mathbf{x}|\omega_i)P(\omega_i)}{p(\mathbf{x})}$$

最小错误率决策：若 $P(\omega_i|\mathbf{x}) > P(\omega_j|\mathbf{x})$ ，则判定 $\mathbf{x} \in \omega_i$ 。

多元正态分布 (Gaussian Density)：

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp \left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}) \right]$$

线性判别函数 (Linear Discriminant)：

$$g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$$

最大似然估计 (MLE)：

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta}} \prod_{k=1}^N p(\mathbf{x}_k|\boldsymbol{\theta}) \quad \Rightarrow \quad \hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta}} \sum_{k=1}^N \ln p(\mathbf{x}_k|\boldsymbol{\theta})$$

SVM 优化目标 (硬间隔)：

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, N$$

特征值分解 (PCA)： 协方差矩阵 $\mathbf{S}$ 的特征方程：

$$\mathbf{S} \mathbf{u}_i = \lambda_i \mathbf{u}_i$$

K-Means 目标函数：

$$J = \sum_{j=1}^K \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \mathbf{m}_j\|^2$$